

Práctica. Backend API en Node.js.

Instrucciones. Elabore una aplicación web de Node.js utilizando el patrón de diseño de software MVC con modelos y controladores con autorización, que implemente el método de autenticación JWT, el manejo de archivos, una bitácora de acciones, además del manejo y registro de errores.

1. Sigue las instrucciones de la práctica en la siguiente página.
2. Crea un reporte de práctica en un **archivo de Word** con una portada con tu nombre. Guárdalo con el nombre **NombreAlumno_BackendNode.docx**.
3. Coloca **las siguientes capturas de pantalla**, cada una con una **descripción textual** arriba de la captura.
 - a. Coloca capturas de pantalla del código fuente donde se exponga la funcionalidades de la aplicación.
 - b. Coloca capturas de pantalla de Visual Studio Code con la extensión REST API ejecutando consultas GET, POST, PUT y DELETE.
 - c. Coloca capturas de pantalla de la consola de MySQL listando las tablas con datos en tu base de datos de MySQL creadas por el ORM.
 - d. Publica tu código fuente en un proyecto público de GitHub. **Coloca la URL** del código fuente publicado en GitHub.
4. Sube tu reporte de práctica archivo Word a la actividad en Eminus.

Prerrequisitos

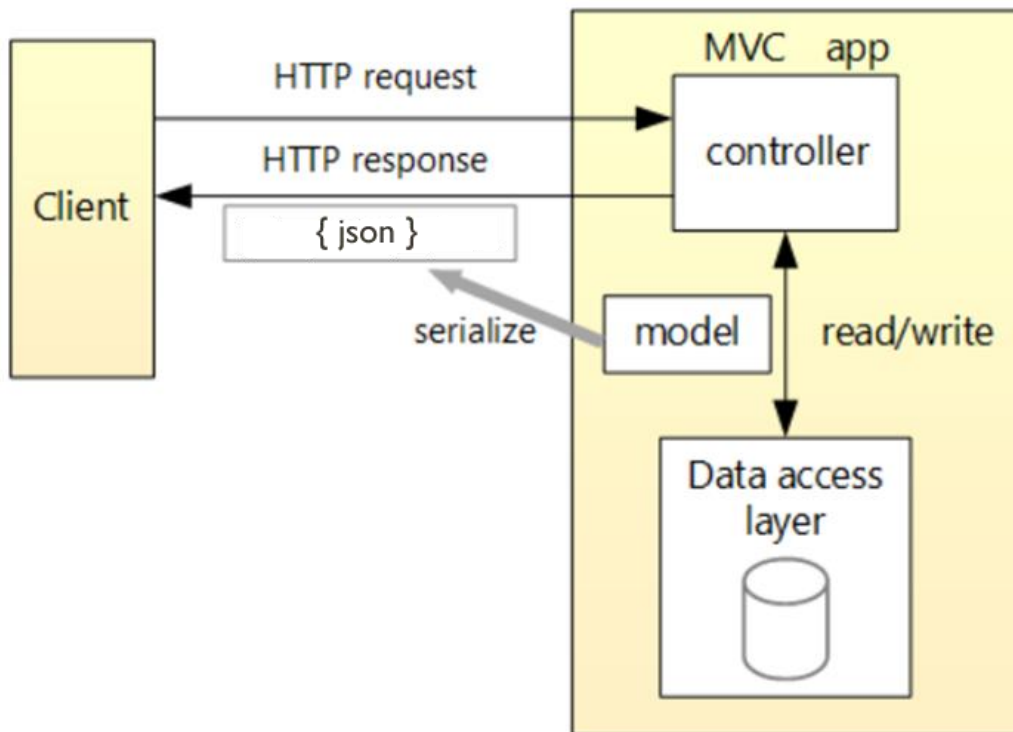
1. Esta práctica tiene como prerrequisito tener instalado el software **Visual Studio Code** con la extensión REST Client.
Puede seguir las instrucciones de cómo instalar Visual Studio Code desde la práctica [Instalación de Visual Studio Code](#).
2. Esta práctica tiene como prerrequisito tener instalado la plataforma Node.js en su equipo.
Puede seguir las instrucciones de cómo instalar Node.js desde la práctica [Instalación de Node.js](#).
3. Esta práctica tiene como prerrequisito tener instalado MySQL en su equipo.
Puede seguir las instrucciones de cómo instalar MySQL desde la práctica [Instalación de MySQL](#).
4. Puede seguir las instrucciones de cómo publicar un proyecto en Visual Studio Code a GitHub desde la práctica [Como publicar un proyecto a GitHub desde Visual Studio Code](#).

Instrucciones. Defina su API.

Lo primero que tenemos que realizar antes de comenzar a escribir código y crear, es definir el API. Esta práctica crea la siguiente API:

API	Descripción	Cuerpo de la solicitud	Cuerpo de la respuesta	Rol requerido
<i>GET</i> /api/archivos	Obtener todas las archivos	Ninguno	Lista de archivos	Administrador
<i>GET</i> /api/archivos/{id}	Obtener una archivo por ID	Ninguno	Archivo	Administrador
<i>POST</i> /api/archivos	Agregar una nueva archivo	Categoría	Archivo agregado	Administrador
<i>PUT</i> /api/archivos/{id}	Actualizar una archivo existente	Categoría	Ninguno	Administrador
<i>DELETE</i> /api/archivos/{id}	Eliminar una archivo	Ninguno	Ninguno	Administrador
<i>GET</i> /api/bitacora	Obtener todas las filas de la bitácora	Ninguno	Lista de operaciones	Administrador
<i>GET</i> /api/categorias	Obtener todas las categorías	Ninguno	Lista de categorías	Administrador, Usuario
<i>GET</i> /api/categorias/{id}	Obtener una categoría por ID	Ninguno	Categoría	Administrador, Usuario
<i>POST</i> /api/categorias	Agregar una nueva categoría	Categoría	Categoría agregada	Administrador
<i>PUT</i> /api/categorias/{id}	Actualizar una categoría existente	Categoría	Ninguno	Administrador
<i>DELETE</i> /api/categorias/{id}	Eliminar una categoría	Ninguno	Ninguno	Administrador
<i>GET</i> /api/productos	Obtener todas las productos	Ninguno	Lista de productos	Administrador, Usuario
<i>GET</i> /api/productos?s=título	Obtener todas las productos que contengan en el título la cadena s	Ninguno	Lista de productos	Administrador, Usuario
<i>GET</i> /api/productos/{id}	Obtener una producto por ID	Ninguno	Producto	Administrador, Usuario
<i>POST</i> /api/productos	Agregar una nueva producto	Producto	Producto agregada	Administrador
<i>PUT</i> /api/productos/{id}	Actualizar una producto existente	Producto	Ninguno	Administrador
<i>DELETE</i> /api/productos/{id}	Eliminar una producto	Ninguno	Ninguno	Administrador
<i>GET</i> /api/usuarios	Obtener todas los usuarios	Ninguno	Lista de usuarios	Administrador
<i>GET</i> /api/usuarios/{id}	Obtener un usuario por ID	Ninguno	Usuario	Administrador
<i>POST</i> /api/usuarios	Agregar un nuevo usuario	Usuario	Usuario agregado	Administrador
<i>PUT</i> /api/usuarios/{id}	Actualizar un usuario existente	Usuario	Ninguno	Administrador
<i>DELETE</i> /api/usuarios/{id}	Eliminar un usuario	Ninguno	Ninguno	Administrador
<i>GET</i> /api/roles	Obtener todas los roles	Ninguno	Lista de roles	Administrador
<i>GET</i> /api/auth	Inicio de sesión	Ninguno	JWT	Ninguno
<i>GET</i> /api/tiempo	Obtener el tiempo restante de sesión	Ninguno	JWT	Administrador, Usuario

El siguiente diagrama muestra el diseño de la aplicación.



¿Qué es un Json Web Token?

Un JSON Web Token (JWT) es un estándar ([RFC 7519](#)) que define una forma segura y compacta de transmitir información entre dos entidades en forma de un objeto JSON.

Esta información puede ser verificada y es confiable ya que está firmada digitalmente. Los JWTs pueden ser firmados utilizando una llave privada (con un algoritmo [HMAC](#)) o con llaves públicas y privadas utilizando [RSA](#) o [ECDSA](#).

La autenticación basada en tokens carece de estado (es stateless).

¿Cuándo se debería utilizar Json Web Tokens?

Aquí veremos un par de escenarios donde es útil y recomendable utilizar los JWTs:

- **Autorización:** Este es el caso de uso más común de los JWTs. Una vez que un usuario ha iniciado sesión, cada llamada subsecuente al servicio incluirá el JWT, permitiendo al usuario acceder a rutas, servicios o recursos que solo están permitidos con su debido token. SSO (Single Sign On) es una funcionalidad que hoy en día usa los JWTs ampliamente, porque son de tamaño reducido y por su habilidad de ser usado entre diferentes dominios.
- **Intercambio de Información:** Los JWTs son útiles también para transmitir información entre dos entidades. Debido a que los JWTs pueden estar firmados — por ejemplo, utilizando una llave pública/privada — podemos estar seguros que quien manda la información es verdaderamente él quien lo manda. Adicionalmente, la firma es calculada utilizando el encabezado del JWT y el contenido (payload) por lo que también estamos seguros que el contenido del JWT no fue alterado.

¿Qué estructura tiene un JWT?

Un JWT está separado por puntos (.) en tres partes, las cuales son:

1. Encabezado (**header**)
2. Contenido (**payload**)
3. Firma (**signature**)

Un JWT comúnmente tiene la siguiente forma.

xxxxx.yyyyy.zzzzz

Veamos que significa cada una de estas partes.

1. Header

El encabezado *típicamente* consiste de dos partes: el tipo de token (que será JWT) y el algoritmo que se está usando en la firma, que puede ser **HMAC SHA256** o **RSA**.

Por ejemplo:

```
{  
  "alg": "http://www.w3.org/2001/04/xmldsig-more#hmac-sha256",  
  "typ": "JWT"  
}
```

Después, este JSON se codifica en **Base64URL** para formar parte del primer segmento del JWT.

2. Payload

La segunda parte del JWT es el contenido que se transmite o certifica (payload), el cual contiene la serie de claims. Claims son afirmaciones sobre una entidad (usualmente, el usuario) e información adicional. Hay tres tipos de claims: registrados, públicos y privados.

- **Claims registrados:** Son un conjunto de claims predefinidos que no son obligatorios pero sí recomendados, para proveer un conjunto de claims interoperables. Algunos de ellos son: **iss** (issuer), **exp** (tiempo de expiración), **sub** (subject), **aud** (audience), entre [otros](#).

Nótese que los nombres de los claims son de tres letras por la misma intención de mantener el JWT de tamaño reducido.

- **Claims públicos:** Estos pueden ser definidos como cada quien desee, pero para evitar colisiones de nombres y mantener un estándar (ya que puede usarse en distintos servicios), se utiliza la siguiente lista llamada [IANA JSON Web Token Registry](#).
- **Claims privados:** Estos claims son personalizados por cada quien que implemente los JWTs y al igual que los públicos, para evitar colisiones es recomendable utilizar un formato URL con algún namespace y así asegurar que son únicos
 - Por ejemplo, un claim que guarda los roles de ASP.NET Core tendría el siguiente nombre: `http://schemas.microsoft.com/ws/2008/06/identity/claims/role`.

Un ejemplo de un payload sería el siguiente:

```
{
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/sid": "09e30209-acbf-4391-baea-c719dd271c5f",
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/name": "gvera@uv.mx",
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/emailaddress": "gvera@uv.mx",
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/givenname": "Guillermo Humberto Vera Amaro",
  "http://schemas.microsoft.com/ws/2008/06/identity/claims/role": "Administrador",
  "exp": 1701147212,
  "iss": "ServidorFeiJWT",
  "aud": "ClienteFeiJWT"
}
```

Y al igual que el header, este segmento se codifica en **Base64Url**.

Nota: Aunque los JWT estén firmados, solo están protegidos para evitar falsificaciones (editar el payload) pero de igual forma, toda la información en el payload es visible para cualquiera. **NO se recomienda incluir información sensible en el payload** al menos que esté encriptada*.*

3. Signature

Para crear la firma debemos de tomar el header codificado, el payload codificado, una llave secreta, el algoritmo especificado en el header y firmar todo eso.

Por ejemplo, si vamos a utilizar el algoritmo de encriptación HMAC SHA256, la firma será creada de la siguiente forma:

```
HMACSHA256(base64UrlEncode(header) + "." + base64UrlEncode(payload), secret)
```

La firma se usará para verificar que el mensaje no ha cambiado mientras viaja por la red, y en caso de ser tokens firmados por una llave privada de un certificado, también se puede verificar el emisor.

Juntando todo

Al final, tendremos tres cadenas de texto codificadas en Base64-URL separadas por puntos y se podrán incluir en solicitudes HTTP o contenido HTML sin ningún problema. Esto es una forma mucho más compacta comparado a otros estándares como SAML que utiliza XML.

Al final, tendríamos un JWT de la siguiente forma:

```
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiIxMjM0NTY3ODkwIiwibmFtZSI6IkpvaG4gRG91IiwiaWF0IjoxNTE2MjM5MDIyfQ.SflKxwRJSMeKKF2QT4fwpMeJf36P0k6yJV_adQssw5c
```

Para verificar JWT, se puede visitar el sitio jwt.io.

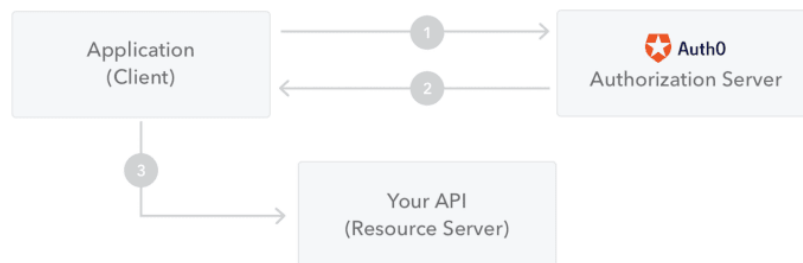
¿Cómo funcionan los JWT?

Cuando un usuario ha sido autenticado, el servicio deberá regresar un JSON Web Token para ser usado como sus credenciales. Dado que esto es usado para autorizar el usuario, debes de considerar cuidar muy bien donde guardas el token, y eliminarlo lo más pronto posible si ya no se requiere.

Cuando un usuario quiere acceder a contenido restringido en una ruta protegida, se debe de incluir el token en el HTTP Header **Authorization** y utilizando el esquema **Bearer**.

Ejemplo:

```
HeaderNames.Authorization, "Bearer " + accessToken
```



Generalmente en Web APIs (y como lo haremos más adelante) que son aplicaciones stateless, siempre requerirá que el token vaya incluido en el encabezado Authorization. El servicio verificará lo necesario para determinar si es un token válido o no, y si este es válido. leerá su información (los claims) y lo usará en la solicitud de ser necesario.

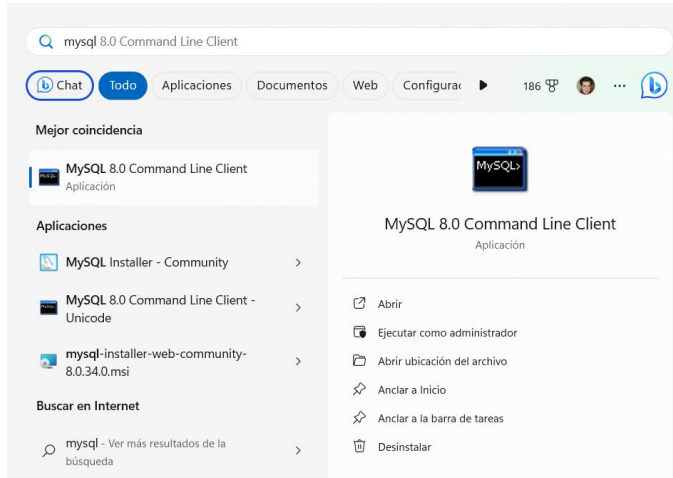
Esto también reduce las consultas a bases de datos para leer información del usuario, ya que el token puede contener información común para poder operar (como username, email, roles, etc).

Dado que el token va incluido en el header, no habrá problemas con el Cross-Origin Resource Sharing (CORS) ya que no se utilizan cookies (las cookies son por dominio).

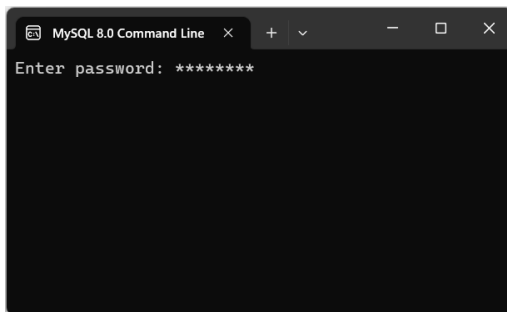
1. La aplicación cliente solicita autorización al Identity Server (como el que vamos a utilizar).
2. Cuando se autoriza el acceso, el servidor de autorización regresa el access token a la aplicación cliente.
3. La aplicación cliente usa el access token para acceder a recursos protegidos (como los endpoints de una API).

Instrucciones. Crear la base de datos

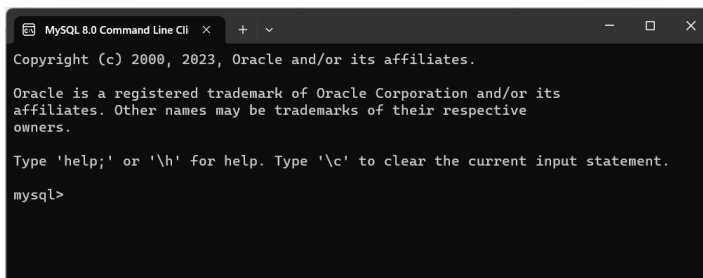
1. Usted ya debería contar con esta base de datos del proyecto anterior. Solo realice el paso de eliminarla y volverla a crear para tenerla completamente vacía.
2. En el menú de inicio de Windows, busque el programa MySQL escribiendo `mysql` y haga clic en ejecutar. También puede ejecutar la versión “MySQL 8.0 Command Line Client Unicode” que tiene soporte para acentos y caracteres especiales como la letra ñ.



3. Escriba la contraseña del super usuario `root` que creó en la instalación de MySQL.



4. Si se equivoca en la contraseña, la ventana se cerrará y tendrá que volver a abrir el programa cliente.
5. El cliente de MySQL debe verse de la siguiente forma. Esto quiere decir que se ha conectado correctamente a su servidor local MySQL con el usuario `root`.



6. Abra Visual Studio Code.
7. Asegúrese de contar con una carpeta de trabajo en `C:\codigo`. Si aún no ha creado la carpeta `C:\codigo` hágalo antes de continuar.
8. Seleccione **Archivo** > **Abrir carpeta** en el menú principal.

9. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\backendnode** y selecciónela. Luego haga clic en Seleccionar carpeta.
10. Cree un nuevo archivo llamado **bd.sql** para crear la base de datos de la aplicación. Agregué las siguientes instrucciones al archivo.

```
DROP DATABASE IF EXISTS mercadolibrenode;  
CREATE DATABASE mercadolibrenode CHARACTER SET utf8mb4;  
USE mercadolibrenode;
```

11. Dentro de MySQL, para ejecutar el script de creación de la base de datos, ejecute las siguiente instrucción **source**:

```
mysql> source C:\codigo\backendnode\bd.sql
```

12. Verifique que se la base de datos se haya creado.

```
mysql> show databases;  
+-----+  
| Database |  
+-----+  
| information_schema |  
| mysql |  
| mercadolibrenode |  
| performance_schema |  
| sys |  
+-----+  
5 rows in set (0.00 sec)
```

13. Ahora vamos a crear el usuario para conectarse desde su aplicación. En este ejemplo le colocaremos el nombre **mercadolibre_user** con password **M3rc@d0**, pero usted puede colocar el que prefiera.

```
mysql> CREATE USER mercadolibre_user@localhost IDENTIFIED BY 'M3rc@d0';  
Query OK, 0 rows affected (0.01 sec)
```

14. Y para darle permisos en la nueva base de datos, se utiliza:

```
mysql> GRANT ALL ON mercadolibrenode.* TO mercadolibre_user@localhost;  
Query OK, 0 rows affected (0.01 sec)
```

15. Si solo quisiéramos darle permisos de lectura y escritura, sin que pudiera crear objetos, se usaría:

```
mysql> GRANT SELECT, INSERT, UPDATE, DELETE ON mercadolibrenode.* TO mercadolibre_user@localhost;
```

16. No utilice este comando en este momento, pues queremos que el usuario si pueda crear, modificar y eliminar las tablas que serán parte de la base de datos. Después de la creación de las tablas, puede revocar los privilegios y utilizar este comando para un entorno más seguro.

17. Si quisiera revocar los permisos del usuario se podría usar:

```
mysql> REVOKE ALL ON mercadolibrenode.* FROM mercadolibre_user@localhost;
```

18. Para verificar que los permisos se hayan aplicado se ejecuta:

```
mysql> SHOW GRANTS FOR mercadolibre_user@localhost;  
+-----+  
| Grants for mercadolibre_user@localhost |
```

```
+-----+
| GRANT USAGE ON *.* TO `mercadolibre_user`@`localhost` |
| GRANT ALL PRIVILEGES ON `mercadolibrenode`.* TO `mercadolibre_user`@`localhost` |
+-----+
2 rows in set (0.00 sec)
```

19. Puede cerrar la sesión con `exit;` y salir.

Instrucciones: Creación de la aplicación API.

1. Asegúrese de contar con una carpeta de trabajo en **C:\codigo**. Si aún no ha creado la carpeta **C:\codigo** hágalo antes de continuar.
2. Desde VS Code seleccione **Archivo > Abrir carpeta** en el menú principal.
3. En el cuadro de diálogo **Abrir carpeta**, cree una carpeta en la ruta **C:\codigo\backendnode** y selecciónela. Luego haga clic en Seleccionar carpeta.
4. El nombre de la carpeta se convierte en el nombre del proyecto y el nombre del espacio de nombres de forma predeterminada.
5. Abra una Terminal de consola presionando **Ctrl + ñ**.
6. Asegúrese de tener instalado Node.js en su equipo. En caso de que el comando no sea reconocido, instale Node.js desde la práctica [Instalación de Node.js](#).

```
> node -v  
v20.16.0
```

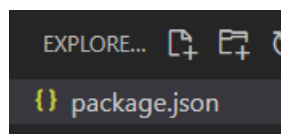
7. Utilice el comando **npm init** para crear un archivo **package.json** para su aplicación.

```
> npm init
```

8. Este comando le solicita varias cosas, como el nombre y la versión de su aplicación. Puede simplemente presionar **Enter** para aceptar los valores predeterminados.

```
package name: (backendnode)  
version: (1.0.0)  
description:  
entry point: (index.js)  
test command:  
git repository:  
keywords:  
author:  
license: (ISC)  
About to write to C:\codigo\backendnode\package.json:  
  
{  
  "name": "backendnode",  
  "version": "1.0.0",  
  "description": "",  
  "main": "index.js",  
  "scripts": {  
    "test": "echo \"Error: no test specified\" && exit 1"  
  },  
  "author": "",  
  "license": "ISC"  
}  
Is this OK? (yes)
```

9. Si quisiera omitir el asistente, solo se agrega la flag **-y** al comando **npm init -y**.
10. Observe que se ha creado un archivo llamado **package.json**. Deje abierto este archivo.



11. Ahora instale el paquete **Express** en este directorio y guárdelo en la lista de dependencias del proyecto.

```
> npm install express --save
```

12. Observe que se ha instalado una versión de Express, en este caso la versión **4.19.2**.

```
{
  "name": "backendnode",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "start": "nodemon index.js",
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "dependencies": {
    "express": "^4.19.2"
  }
}
```

13. También instale el paquete **dotenv** que permite cargar variables de entorno desde un archivo con extensión **.env** hacia **process.env**. En este caso, utilizamos **dotenv** para almacenar las configuraciones de la base de datos y el puerto del servidor.

```
npm install dotenv --save
```

14. Ahora instale el paquete **cors** que permite a clientes de otros dominios o puertos, conectarse a los **endpoints** de esta API.

```
npm install cors --save
```

15. Ahora instale el paquete **mysql2** que es el cliente MySQL para Node.js con enfoque en el rendimiento.

```
npm install mysql2 --save
```

16. Solo como referencia, para instar un paquete temporalmente sin agregarlo a la lista de dependencias, se puede agregar **--no-save**.

17. Solo como referencia, para desinstalar un paquete, se utiliza **npm uninstall <package_name>**.

18. Ahora vamos a crear el archivo **.env** en la raíz de su aplicación. Una vez creado, agregue los datos de su conexión a MySQL.

```
DB_HOST=localhost
DB_DATABASE=mercadolibrenode
DB_USER=mercadolibre_user
DB_PASSWORD=M3rc@d0
DB_PORT=3306
SERVER_PORT=3000
NODE_ENV=development
JWT_SECRET=865ad24cf96fa78ab5c3cdb48f5fb114923d95c8a4e4f1bc9c33f6eeda32ed9d4b8c57
FILES_IN_DB=false
```

19. No se preocupe por los valores de las variables `JWT_SECRET` y `FILES_IN_DB`, estas se verán más adelante.
20. Ahora vamos a crear el archivo principal de Node.js. Cree un nuevo archivo llamado `index.js` y agregue el siguiente código.

```
const express = require('express')
const dotenv = require('dotenv')
const app = express()

// Primero carga la configuración del archivo .env para que este disponible en las demás llamadas
dotenv.config()

app.get("/", (req, res) => {
  res.send("Hola mundo!")
})

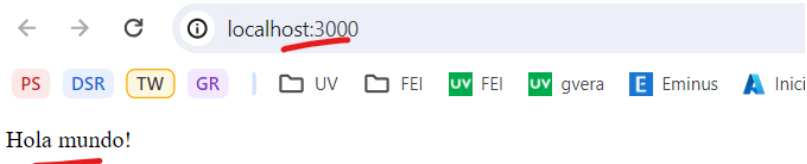
// Inicia el servidor web en el puerto SERVER_PORT
app.listen(process.env.SERVER_PORT, () => {
  console.log(`Aplicación de ejemplo escuchando en el puerto ${process.env.SERVER_PORT}`)
})
```

21. Ejecute la aplicación con el siguiente comando desde la terminal de VS Code. Verifique que su aplicación no marque errores.

```
> node index.js
```

```
PS C:\codigo\tw\mvcnode> node index.js
Aplicación de ejemplo escuchando en el puerto 3000
```

22. Abra un navegador web y abra la ruta <http://localhost:3000/> para ver el resultado.



The screenshot shows a web browser window with the address bar set to `localhost:3000`. Below the address bar, there are several tabs labeled with initials: PS, DSR, TW, GR, UV, FEI, FEI, gvera, Eminus, and Inici. The main content area of the browser displays the text `Hola mundo!` in a red font, which is underlined.

23. Para detener el servidor, solo presione **Ctrl + C** en la consola.
24. Como cada vez que realice un cambio en su código, debe repetir esta operación, puede utilizar el paquete **nodemon**. Se puede instalar como una dependencia de desarrollo con **--save-dev** y así no se copiará a producción; pues solo es un paquete que se usa en desarrollo.

```
npm install nodemon --save-dev
```

25. Observe en su archivo **package.json** que se agrega en una lista aparte de dependencias para desarrollo.

```
"devDependencies": {  
  "nodemon": "^3.1.0"  
}
```

26. En este mismo archivo **package.json**, debajo del apartado de scripts, agregue un nombre de script para ejecutar **nodemon** cada vez que escribamos **node start** en la consola.

```
"scripts": {  
  "start": "nodemon index.js",  
}
```

27. Ahora puede utilizar **npm start** para iniciar su aplicación. Observe la consola.

```
PS C:\codigo\tw\mvcnode> npm start  
  
> mvcnode@1.0.0 start  
> nodemon index.js  
  
[nodemon] 3.1.0  
[nodemon] to restart at any time, enter `rs`  
[nodemon] watching path(s): *.*  
[nodemon] watching extensions: js,mjs,cjs,json  
[nodemon] starting `node index.js`  
Aplicación de ejemplo escuchando en el puerto 3000
```

28. Ahora cuando realice cualquier cambio en el código, el servidor se reiniciará de manera automática.
29. Cambie el mensaje en el método **get**.

```
app.get("/", (req, res) => {  
  res.send("Hola mundo con nodemon!")  
})
```

30. Observe como se reinicia el servidor y puede ver los cambios de inmediato al refrescar su navegador.

```
Aplicación de ejemplo escuchando en el puerto 3000  
[nodemon] restarting due to changes...  
[nodemon] starting `node index.js`  
Aplicación de ejemplo escuchando en el puerto 3000
```

31. Ya que ha practicado el cómo instalar paquetes, instale de la misma manera los paquetes `bcrypt`, `cors`, `dotenv`, `express`, `jsonwebtoken`, `multer`, `mysql2`, `request-ip` y el paquete `swagger-ui-express` como dependencias.
32. Igualmente verifique instalar el paquete `nodemon`, y el paquete `swagger-autogen` como dependencias solo para desarrollo.

```
"dependencies": {
  "bcrypt": "^5.1.1",
  "cors": "^2.8.5",
  "dotenv": "^16.4.5",
  "express": "^4.19.2",
  "express-validator": "^7.1.0",
  "jsonwebtoken": "^9.0.2",
  "multer": "^1.4.5-lts.1",
  "mysql2": "^3.9.3",
  "request-ip": "^3.3.0",
  "swagger-ui-express": "^5.0.0"
},
"devDependencies": {
  "nodemon": "^3.1.0",
  "swagger-autogen": "^2.23.7"
}
```

33. Para terminar la ejecución escriba en la consola `ctrl + c` en Visual Studio Code.
34. Felicidades, ha creado su aplicación de manera correcta.

Instrucciones: Agregar la funcionalidad ORM

1. Abra una terminal de consola con `Ctrl + ñ`.
2. Instale el paquete para darle el soporte de **ORM** a su aplicación. En Node el **ORM** se llama **Sequelize** y funciona para Postgres, MySQL, MariaDB, SQLite y SQL Server. Puede encontrar más información desde [Sequelize](#).

```
npm install sequelize --save
```

3. Agregue el paquete para poder hacer **Migrations**. Las migraciones le permiten mantener sincronizados sus modelos con su base de datos.

```
npm install -g sequelize-cli
```

4. Agregue el paquete para utilizar MySQL con **ORM**.

```
npm install mysql2 --save
```

5. Después de la instalación, utilice CLI para generar migraciones, seeders, configuración, modelos y el archivo de configuración. En la terminal ejecute el siguiente comando utilizando npx (Node Package Execute) que nos va a ayudar a ejecutar el paquete **sequelize-cli**.

```
npx sequelize init
```

6. El comando se puede escribir solo **sequelize** o **sequelize-cli**.
7. Observe con solo escribir **sequelize**, se manda llamar **sequelize-cli**. Observe las carpetas creadas.

```
PS C:\codigo\tw\backendnode> npx sequelize init
```

```
Sequelize CLI [Node: 20.11.0, CLI: 6.6.2, ORM: 6.37.2]
```

```
Created "config\config.json"
```

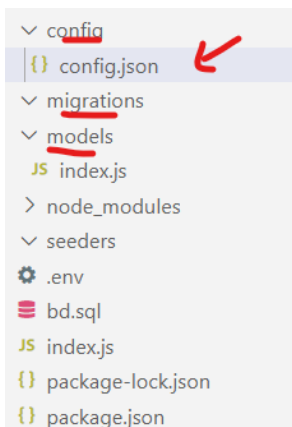
```
Successfully created models folder at "C:\codigo\tw\backendnode\models".
```

```
Successfully created migrations folder at "C:\codigo\tw\backendnode\migrations".
```

```
Successfully created seeders folder at "C:\codigo\tw\backendnode\seeders".
```

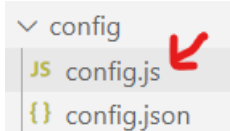
```
PS C:\codigo\tw\backendnode> 
```

8. Si inspecciona su directorio en este momento, se dará cuenta de que el comando anterior acaba de crear los directorios **config**, **models**, **migrations**, **seeders** y generó el código en ellos.



9. Ahora se va a configurar la conexión a la base de datos. El archivo `config/config.json` que se encuentra en `json`, sirve para crear una configuración **estática**. Como se desea una configuración **dinámica** leyendo valores desde el archivo `.env`, debe crear un nuevo archivo en `config/config.js` (observe que la extensión es distinta). Entonces, en la carpeta `config` le quedaran dos archivos: `config.json` (aunque no se usará, no lo borre) y `config.js` con el siguiente contenido.

```
module.exports = {
  development: {
    username: process.env.DB_USER,
    password: process.env.DB_PASSWORD,
    database: process.env.DB_DATABASE,
    host: process.env.DB_HOST,
    port: process.env.DB_PORT,
    dialect: 'mysql'
  },
  test: {
    username: process.env.TEST_DB_USER,
    password: process.env.TEST_DB_PASSWORD,
    database: process.env.TEST_DB_DATABASE,
    host: process.env.TEST_DB_HOST,
    port: process.env.TEST_DB_PORT,
    dialect: 'mysql'
  },
  production: {
    username: process.env.PROD_DB_USERNAME,
    password: process.env.PROD_DB_PASSWORD,
    database: process.env.PROD_DB_NAME,
    host: process.env.PROD_DB_HOSTNAME,
    port: process.env.PROD_DB_PORT,
    dialect: 'mysql'
  }
}
```



10. Por el momento solo utilizaremos la sección llamada `development` pero usted puede utilizar las otras secciones o crear nuevas dependiendo de las necesidades de sus proyectos.
11. Y ahora para decirle a `sequelize` del cambio, debemos crear un archivo en raíz llamado `.sequelizerc` con el siguiente contenido.

```
// .sequelizerc

require('dotenv').config()
const path = require('path')

module.exports = {
  'config': path.resolve('config', 'config.js')
}
```

12. También debemos modificar el archivo `/models/index.js` avisando del cambio.

```
'use strict';

const fs = require('fs');
const path = require('path');
const Sequelize = require('sequelize');
const process = require('process');
const basename = path.basename(__filename);
const env = process.env.NODE_ENV || 'development';
// const config = require(__dirname + '/../config/config.json')[env];
const config = require('../config/config')[env];
const db = {};
```

13. Finalmente, asegúrese de crear las siguientes carpetas en el proyecto:

- **config.** Aquí van la configuraciones.
- **controllers.** Aquí van los controladores.
- **log.** Aquí van los archivos de logging.
- **middlewares.** Aquí va funcionalidad extra que puede utilizar la aplicación.
- **migrations.** Aquí se colocan las migraciones.
- **models.** Aquí van los modelos.
- **restclient.** Aquí van las consultas de ayuda de la extensión restclient.
- **routes.** Aquí van la definición de las rutas que llaman a los controladores.
- **seeders.** Aquí van la inicialización de los datos.
- **services.** Aquí van los componentes reutilizables de código.
- **uploads.** Aquí van los archivos subidos si decidimos colocarlos en el file system.

14. Felicidades, ha configurado correctamente su aplicación.

Instrucciones. Agregar los Modelos.

1. Se utilizará el comando `sequelize-cli` para generar archivos de modelo y migraciones.
2. Se deben de crear los modelos: `categoria`, `producto`, la relación muchos a muchos entre ellos llamada `categoriaproducto`, además `usuario` y `rol`, y finalmente los modelos `archivo` y `bitacora`.
3. Para crear el modelo `categoria`, ejecute el siguiente comando.

```
npm run sequelize model:generate --name categoria --attributes nombre:string,protegida:boolean
```

```
PS C:\codigo\tw\backendnode> npm run sequelize model:generate --name categoria --attributes nombre:string,protegida:boolean
```

```
Sequelize CLI [Node: 20.11.0, CLI: 6.6.2, ORM: 6.37.2]
```

```
New model was created at C:\codigo\tw\backendnode\models\categoria.js .
```

```
New migration was created at C:\codigo\tw\backendnode\migrations\20240330022819-create-categoria.js .
```

4. Como resultado deberíamos tener en `models` el archivo `categoria.js` y en `migrations` otro archivo llamado `xxxx-create-categoria.js`.
5. No todo lo predeterminado está correcto. Vamos a modificar el archivo `categoria.js` y debe quedar de la siguiente forma.

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class categoria extends Model {
    static associate(models) {
      categoria.belongsToMany(models.producto, { through: 'categoriaproducto', foreignKey: 'categoriaid' });
    }
  }

  categoria.init({
    id: {
      type: DataTypes.INTEGER,
      autoIncrement: true,
      primaryKey: true
    },
    nombre: {
      type: DataTypes.STRING,
      allowNull: false
    },
    protegida: {
      type: DataTypes.BOOLEAN,
      defaultValue: false
    }
  }, {
    sequelize,
    freezeTableName: true,
    modelName: 'categoria',
  });

  return categoria;
};
```

6. Y dentro del nuevo archivo creado `xxxx-create-categoria.js` en la carpeta `migrations`, escribimos el siguiente código.

```
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('categoriaproducto', {
      categoriaid: {
        allowNull: false,
        primaryKey: true,
        type: Sequelize.INTEGER,
        references: {
          model: 'categoria',
          key: 'id'
        },
        onUpdate: 'CASCADE',
        onDelete: 'CASCADE'
      },
      productoid: {
        allowNull: false,
        primaryKey: true,
        type: Sequelize.INTEGER,
        references: {
          model: 'producto',
          key: 'id'
        },
        onUpdate: 'CASCADE',
        onDelete: 'CASCADE'
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE
      }
    });
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable('categoriaproducto');
  }
};
```

7. Para crear el modelo **producto**, ejecute el siguiente comando

```
npm run sequelize model:generate --name producto --attributes
titulo:string,sinopsis:string,anio:integer,poster:string
```

```
PS C:\codigo\tw\backendnode> npm run sequelize model:generate --name pelicula --attributes titulo:string,sinopsis:string,anio:integer,poster:string
```

Sequelize CLI [Node: 20.11.0, CLI: 6.6.2, ORM: 6.37.2]

New model was created at C:\codigo\tw\backendnode\models\pelicula.js .

New migration was created at C:\codigo\tw\backendnode\migrations\20240330024917-create-pelicula.js .

8. Como resultado deberíamos tener en `models` el archivo `producto.js` y en `migrations` otro archivo llamado `xxxx-create-producto.js`.
9. No todo lo predeterminado está correcto. Vamos a modificar el archivo `producto.js` y debe quedar de la siguiente forma.

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class producto extends Model {
    static associate(models) {
      producto.belongsToMany(models.categoria, { as: 'categorias', through: 'categoriaproducto', foreignKey: 'productoid' });
      producto.belongsTo(models.archivo);
    }
  }

  producto.init({
    id: {
      type: DataTypes.INTEGER,
      autoIncrement: true,
      primaryKey: true
    },
    titulo: {
      type: DataTypes.STRING,
      defaultValue: "Sin titulo"
    },
    descripcion: {
      type: DataTypes.TEXT,
      defaultValue: "Sin descripcion"
    },
    precio: {
      type: DataTypes.DECIMAL(10,2),
      allowNull: false
    },
    archivoid: {
      type: DataTypes.INTEGER,
      allowNull: true
    }
  }, {
    sequelize,
    freezeTableName: true,
    modelName: 'producto',
  });

  return producto;
};
```

10. Y dentro del nuevo archivo creado `xxxx-create-producto.js` en la carpeta `migrations`, escribimos el siguiente código.

```
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('producto', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER
      },
      titulo: {
        type: Sequelize.STRING,
        defaultValue: "Sin título"
      },
      descripcion: {
        type: Sequelize.TEXT,
        defaultValue: "Sin descripcion"
      },
      precio: {
        type: Sequelize.DECIMAL(10,2)
      },
      archivoid: {
        allowNull: true,
        type: Sequelize.INTEGER,
        references: {
          model: 'archivo',
          key: 'id'
        },
        onUpdate: 'CASCADE',
        onDelete: 'SET NULL'
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE
      }
    });
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable('producto');
  }
};
```

11. Una vez que los dos modelos están correctos, es hora de agregar el modelo `categoriaproducto` que actúa como relación muchos a muchos entre estas dos. Como `sequelize` no la crea automáticamente, vamos a crear el archivo de migración nosotros.

```
npx sequelize migration:generate --name create-categoriaproducto
```

12. Y dentro del nuevo archivo creado `xxxx-create-categoriaproducto.js` en la carpeta `migrations`, escribimos el siguiente código.

```
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('categoriaproducto', {
      categoriaid: {
        allowNull: false,
        primaryKey: true,
        type: Sequelize.INTEGER,
        references: {
          model: 'categoria',
          key: 'id'
        },
        onUpdate: 'CASCADE',
        onDelete: 'CASCADE'
      },
      productoid: {
        allowNull: false,
        primaryKey: true,
        type: Sequelize.INTEGER,
        references: {
          model: 'producto',
          key: 'id'
        },
        onUpdate: 'CASCADE',
        onDelete: 'CASCADE'
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE
      }
    });
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable('categoriaproducto');
  }
};
```

13. Para crear el modelo `rol`, ejecute el siguiente comando.

```
npm run sequelize model:generate --name rol --attributes id:string,nombre:string
```

14. Como resultado deberíamos tener en `models` el archivo `rol.js` y en `migrations` otro archivo llamado `xxxx-create-rol.js`.

15. No todo lo predeterminado está correcto. Vamos a modificar el archivo `rol.js` y debe quedar de la siguiente forma.

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class rol extends Model {
    static associate(models) {
      rol.hasMany(models.usuario, { foreignKey: 'rolid' });
    }
  }

  rol.init({
    id: {
      type: DataTypes.STRING,
      primaryKey: true
    },
    nombre: {
      type: DataTypes.STRING,
      allowNull: false
    },
  }, {
    sequelize,
    freezeTableName: true,
    modelName: 'rol',
  });
  return rol;
};
```

16. Y dentro del nuevo archivo creado `xxxx-create-rol.js` en la carpeta `migrations`, escribimos el siguiente código.

```
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('rol', {
      id: {
        type: Sequelize.UUID,
        allowNull: false,
        primaryKey: true
      },
      nombre: {
        type: Sequelize.STRING,
        allowNull: false
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE
      }
    });
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable('rol');
  }
};
```


17. Para crear el modelo `usuario`, ejecute el siguiente comando.

```
npx sequelize model:generate --name usuario --attributes
email:string,passwordhash:string,nombre:string,rolid:string
```

18. Como resultado deberíamos tener en `models` el archivo `usuario.js` y en `migrations` otro archivo llamado `xxxx-create-usuario.js`.

19. No todo lo predeterminado está correcto. Vamos a modificar el archivo `usuario.js` y debe quedar de la siguiente forma.

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class usuario extends Model {
    static associate(models) {
      usuario.belongsTo(models.rol);
    }
  }

  usuario.init({
    id: {
      type: DataTypes.STRING,
      primaryKey: true
    },
    email: {
      type: DataTypes.STRING,
      unique: true,
      allowNull: false
    },
    passwordhash: {
      type: DataTypes.STRING,
      allowNull: false
    },
    nombre: {
      type: DataTypes.STRING,
      allowNull: false
    },
    protegido: {
      type: DataTypes.BOOLEAN,
      defaultValue: false
    },
    rolid: {
      type: DataTypes.STRING,
      allowNull: false
    },
  }, {
    sequelize,
    freezeTableName: true,
    modelName: 'usuario',
  });
  return usuario;
};
```

20. Y dentro del nuevo archivo creado `xxxx-create-usuario.js` en la carpeta `migrations`, escribimos el siguiente código.

```
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('usuario', {
      id: {
        type: Sequelize.UUID,
        allowNull: false,
        primaryKey: true
      },
      email: {
        type: Sequelize.STRING,
        unique: true,
        allowNull: false
      },
      passwordhash: {
        type: Sequelize.STRING,
        allowNull: false
      },
      nombre: {
        type: Sequelize.STRING,
        allowNull: false
      },
      protegido: {
        type: Sequelize.BOOLEAN,
        defaultValue: false
      },
      rolid: {
        allowNull: false,
        type: Sequelize.UUID,
        references: {
          model: 'rol',
          key: 'id'
        },
        onUpdate: 'CASCADE',
        onDelete: 'CASCADE'
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE
      }
    });
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable('usuario');
  }
};
```

21. Para crear el modelo `archivo`, ejecute el siguiente comando.

```
npx sequelize model:generate --name archivo --attributes nombre:string,mime:string
```

22. Como resultado deberíamos tener en `models` el archivo `archivo.js` y en `migrations` otro archivo llamado `xxxx-create-archivo.js`.
23. No todo lo predeterminado está correcto. Vamos a modificar el archivo `archivo.js` y debe quedar de la siguiente forma.

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class archivo extends Model {
    static associate(models) {
      archivo.hasMany(models.producto, { foreignKey: 'id' });
    }
  }

  archivo.init({
    id: {
      type: DataTypes.INTEGER,
      autoIncrement: true,
      primaryKey: true
    },
    mime: {
      type: DataTypes.STRING,
      allowNull: false
    },
    nombre: {
      type: DataTypes.STRING,
      allowNull: false
    },
    size: {
      type: DataTypes.INTEGER,
      allowNull: false
    },
    indb: {
      type: DataTypes.BOOLEAN,
      allowNull: false,
      defaultValue: true
    },
    datos: {
      type: DataTypes.BLOB("long"),
      allowNull: true
    }
  }, {
    sequelize,
    freezeTableName: true,
    modelName: 'archivo',
  });

  return archivo;
};
```

24. Y dentro del nuevo archivo creado `xxxx-create-archivo.js` en la carpeta `migrations`, escribimos el siguiente código.

```
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('archivo', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER
      },
      mime: {
        type: Sequelize.STRING,
        allowNull: false
      },
      nombre: {
        type: Sequelize.STRING,
        allowNull: false
      },
      size: {
        type: Sequelize.INTEGER,
        allowNull: false
      },
      indb: {
        type: Sequelize.BOOLEAN,
        allowNull: false,
        defaultValue: true
      },
      datos: {
        type: Sequelize.BLOB("long"),
        allowNull: true,
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE
      }
    });
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable('archivo');
  }
};
```

25. Para crear el modelo `bitacora`, ejecute el siguiente comando.

```
npm run sequelize model:generate --name bitacora --attributes nombre:string,operacion:string
```

26. Como resultado deberíamos tener en `models` el archivo `bitacora.js` y en `migrations` otro archivo llamado `xxxx-create-bitacora.js`.

27. No todo lo predeterminado está correcto. Vamos a modificar el archivo `bitacora.js` y debe quedar de la siguiente forma.

```
'use strict';
const { Model } = require('sequelize');

module.exports = (sequelize, DataTypes) => {
  class bitacora extends Model {
    static associate(models) {
      // Relaciones
    }
  }

  bitacora.init({
    id: {
      type: DataTypes.INTEGER,
      autoIncrement: true,
      primaryKey: true
    },
    accion: {
      type: DataTypes.STRING,
      allowNull: false
    },
    elementoid: {
      type: DataTypes.STRING,
      allowNull: true
    },
    ip: {
      type: DataTypes.STRING,
      allowNull: false
    },
    usuario: {
      type: DataTypes.STRING,
      allowNull: false
    },
    fecha: {
      type: DataTypes.DATE,
      defaultValue: DataTypes.NOW
    },
  }, {
    sequelize,
    freezeTableName: true,
    modelName: 'bitacora',
  });

  return bitacora;
};
```

28. Y dentro del nuevo archivo creado `xxxx-create-bitacora.js` en la carpeta `migrations`, escribimos el siguiente código.

```
'use strict';
/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.createTable('bitacora', {
      id: {
        allowNull: false,
        autoIncrement: true,
        primaryKey: true,
        type: Sequelize.INTEGER
      },
      accion: {
        type: Sequelize.STRING,
        allowNull: false
      },
      elementoid: {
        type: Sequelize.STRING,
        allowNull: true
      },
      ip: {
        type: Sequelize.STRING,
        allowNull: false
      },
      usuario: {
        type: Sequelize.STRING,
        allowNull: false
      },
      fecha: {
        type: Sequelize.DATE,
        defaultValue: Sequelize.fn("NOW")
      },
      createdAt: {
        allowNull: false,
        type: Sequelize.DATE
      },
      updatedAt: {
        allowNull: false,
        type: Sequelize.DATE
      }
    });
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.dropTable('bitacora');
  }
};
```

29. Con los modelos y las migraciones en su lugar, ahora estamos listos para generar los modelos en la base de datos ejecutando las migraciones.
30. Recuerde que usted puede seguir agregando más modelos si los necesita en su proyecto. Solo repita estos mismos pasos para realizarlo.

Instrucciones. Crear las estructuras de la base de datos.

1. Ya que tenemos listos los modelos y los scripts de las migraciones, ahora estamos listos para generar las tablas en la base de datos ejecutando las migraciones.
2. Para hacer esto, ejecutamos el siguiente comando.

```
npx sequelize db:migrate
```

3. Observe en la consola como se crean las tablas en la base de datos.
4. Si en ahora revisa su servidor de MySQL podrá observar que se han creado las siguientes tablas.

```
mysql> SHOW TABLES;
+-----+
| Tables_in_netflixnode |
+-----+
| archivo                |
| bitacora                |
| categoria              |
| categoriaproducto      |
| producto               |
| rol                    |
| sequelizemeta          |
| usuario                |
+-----+
8 rows in set (0.01 sec)
```

5. También genera otra tabla de apoyo llamada **sequelizemeta** que le indica al **ORM** las migraciones realizadas.
6. Pruebe que puede deshacer la migración realizada escribiendo el comando.

```
npx sequelize db:migrate:undo:all
```

7. Si en ahora revisa su servidor de MySQL podrá observar que se han eliminado las tablas.

```
mysql> SHOW TABLES;
+-----+
| Tables_in_netflixnode |
+-----+
| sequelizemeta          |
+-----+
1 row in set (0.00 sec)
```

8. Vuelva a realizar la migración y sus tablas volverán a la base de datos.

```
npx sequelize db:migrate
```

9. Cada vez que realice un cambio en sus modelos, debe volver a crear archivos de código con los cambios para migraciones con diferente nombre para mantener sincronizado su código y su base de datos.
10. Enhorabuena, ha creado correctamente sus modelos.

Instrucciones. Agregar datos de inicio con el ORM.

1. Para agregar datos de inicio vamos a utilizar el ORM llamado **Sequelize**, en este caso, para trabajar con MySQL. Normalmente se acostumbra inicializar la base de datos con datos de los catálogos.
2. Primero vamos a instalar el paquete para crear hash para las contraseñas.

```
npm install bcrypt --save
```

3. Ahora ejecute el siguiente comando para crear datos de inicio para el modelo **categoria**.

```
npm run sequelize seed:generate --name categorias
```

4. Esto genera un nuevo archivo **xxxx-categorias.js** en la carpeta **seeders**. Modifíquelo de la siguiente manera.

```
'use strict';

/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.bulkInsert('categoria', [
      { id: 1, nombre: 'Televisores', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 2, nombre: 'Celulares y tecnología', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 3, nombre: 'Laptops', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 4, nombre: 'Electrodomésticos', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 5, nombre: 'Hogar y muebles', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 6, nombre: 'Moda', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 7, nombre: 'Deportes', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 8, nombre: 'Construcción', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 9, nombre: 'Oficina', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 10, nombre: 'Juguetes', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 11, nombre: 'Bebés', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 12, nombre: 'Salud', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 13, nombre: 'Belleza', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 14, nombre: 'Videojuegos', protegida: true, createdAt: new Date(), updatedAt: new Date() },
      { id: 15, nombre: 'Supermercado', protegida: true, createdAt: new Date(), updatedAt: new Date() }
    ]);
  },

  async down(queryInterface, Sequelize) {
    await queryInterface.bulkDelete('categoria', null, {});
  }
};
```

5. Puede acceder a este código desde [seed-categorias.js](#).
6. Ejecute el siguiente comando para crear datos de inicio para el modelo **producto**.

```
npm run sequelize seed:generate --name productos
```

7. Esto genera un nuevo archivo **xxxx-productos.js** en la carpeta **seeders**. Modifíquelo de la siguiente manera.

```
'use strict';

/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    // TODO: Implementar la creación de productos de prueba
    // Ejemplo de estructura de datos:
    // await queryInterface.bulkInsert('producto', [
    //   { nombre: 'Producto de prueba', precio: 100, createdAt: new Date(), updatedAt: new Date() },
    //   // ... más productos ...
    // ]);
  },

  async down(queryInterface, Sequelize) {
    // TODO: Implementar la eliminación de productos de prueba
    // Ejemplo de estructura de datos:
    // await queryInterface.bulkDelete('producto', null, {});
  }
};
```


8. Puede acceder a este código desde [seed-productos.js](#).
9. Ejecute el siguiente comando para crear datos de inicio para el modelo `categoriaproducto`.

```
npm run sequelize seed:generate --name categoriaproducto
```

10. Esto genera un nuevo archivo `xxxx-categoriaproducto.js` en la carpeta `seeders`. Modifíquelo de la siguiente manera.

```
'use strict';

/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    await queryInterface.bulkInsert('categoriaproducto', [
      { productoid: 1, categoriaid: 1, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 2, categoriaid: 2, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 3, categoriaid: 3, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 4, categoriaid: 4, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 5, categoriaid: 5, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 6, categoriaid: 6, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 7, categoriaid: 7, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 8, categoriaid: 8, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 9, categoriaid: 9, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 10, categoriaid: 10, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 11, categoriaid: 11, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 12, categoriaid: 12, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 13, categoriaid: 13, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 14, categoriaid: 14, createdAt: new Date(), updatedAt: new Date() },
      { productoid: 15, categoriaid: 15, createdAt: new Date(), updatedAt: new Date() },
    ]),
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.bulkDelete('categoriaproducto', null, {});
  }
};
```

11. Puede acceder a este código desde [seed-categoriaproducto.js](#).
12. Ahora ejecute el siguiente comando para crear datos de inicio para el modelo `usuario`.

```
npm run sequelize seed:generate --name usuarios
```

13. Esto genera un nuevo archivo `xxxx-usuarios.js` en la carpeta `seeders`. Modifíquelo de la siguiente manera.

```
'use strict';
const bcrypt = require('bcrypt');
const crypto = require('crypto');

/** @type {import('sequelize-cli').Migration} */
module.exports = {
  async up(queryInterface, Sequelize) {
    const AdministradorUUID = crypto.randomUUID();
    const UsuarioUUID = crypto.randomUUID();

    await queryInterface.bulkInsert('rol', [
      { id: AdministradorUUID, nombre: 'Administrador', createdAt: new Date(), updatedAt: new Date() },
      { id: UsuarioUUID, nombre: 'Usuario', createdAt: new Date(), updatedAt: new Date() }
    ]);

    await queryInterface.bulkInsert('usuario', [
      { id: crypto.randomUUID(), email: 'gvera@uv.mx', passwordhash: await bcrypt.hash('patito', 10), nombre: 'Guillermo Vera', rolid: AdministradorUUID, protegido: true, createdAt: new Date(), updatedAt: new Date() },
      { id: crypto.randomUUID(), email: 'patito@uv.mx', passwordhash: await bcrypt.hash('patito', 10), nombre: 'Usuario patito', rolid: UsuarioUUID, createdAt: new Date(), updatedAt: new Date() }
    ]);
  },
  async down(queryInterface, Sequelize) {
    await queryInterface.bulkDelete('rol', null, {});
    await queryInterface.bulkDelete('usuario', null, {});
  }
};
```

14. Puede acceder a este código desde [seed-usuarios.js](#).
15. Una vez que tiene definidos su datos de inicio, ejecute el comando para insertarlos en la base de datos. La opción `--debug` le detalla el error en caso que lo hubiera.

```
npx sequelize db:seed:all --debug
```

16. Si desea revertir estas inserciones, ejecute el comando.

```
npx sequelize db:seed:undo:all
```

17. Y para regresar nuevamente los datos, vuelva a ejecutar el comando de `seed`.
18. Con este código, podremos generar nuestra base de datos utilizando el ORM que proporciona `sequelize`.
19. Un **ORM** ofrece varios beneficios, pues el mapeo de las tablas relacionales a objetos de programación, facilita la comunicación con el repositorio, simplifica las consultas, además que agrega seguridad a la aplicación.
20. Puede verificar la creación de sus registros en la consola de MySQL.
21. Enhorabuena, la conexión con su base de datos utilizando un **ORM** esta correctamente realizada.

Instrucciones. Agregar middlewares.

1. El proyecto debe contar con una carpeta llamada `middlewares` y `config`.
2. Primero agregue un nuevo archivo llamado `claimtypes.js` en la carpeta `config` con el siguiente código.

```
const ClaimTypes = {  
  Name: 'http://schemas.xmlsoap.org/ws/2005/05/identity/claims/name',  
  GivenName: 'http://schemas.xmlsoap.org/ws/2005/05/identity/claims/givenname',  
  Role: 'http://schemas.microsoft.com/ws/2008/06/identity/claims/role'  
}  
  
module.exports = ClaimTypes
```

3. Para recibir los archivos adjuntos, deberá utilizar un middleware de node.js como el paquete `Multer` para el manejo de `multipart/form-data`, que se utiliza principalmente para cargar archivos desde formularios.
4. Si aún no lo ha hecho, instale la dependencia desde la consola de Visual Studio Code.

```
npm install multer --save
```

5. Para el caso del middleware que guardara en una bitácora las acciones, vamos a necesitar obtener la IP de los clientes. Para ello instale el paquete `request-ip`.

```
npm install request-ip --save
```

6. En la carpeta `middlewares` agregue un archivo llamado `errorhandler.middleware.js` con el siguiente código.

```
const fs = require("fs")  
const requestIp = require('request-ip');  
const ClaimTypes = require('../config/claimtypes')  
  
const errorHandler = (err, req, res, next) => {  
  let mensaje = 'No se ha podido procesar la petición. Inténtelo nuevamente más tarde.'  
  const statusCode = err.statusCode || 500  
  // Obtiene la IP de la petición  
  const ip = requestIp.getClientIp(req)  
  
  // Se obtiene el mail del usuario actual  
  let email = "Anónimo"  
  if( req.decodedToken ){  
    email = req.decodedToken[ClaimTypes.Name];  
  }  
  // Se guarda en un archivo de texto  
  fs.appendFile('log/log.txt', new Date() + ' - ${statusCode} - ${ip} - ${email} - ${err.message || mensaje}\\n', err => {  
    if (err) {  
      console.error(err);  
    }  
  });  
  
  // Se envía el mensaje apropiado al usuario  
  if (process.env.NODE_ENV === 'development') {  
    mensaje = err.message || mensaje  
    res.status(statusCode).json({  
      status: statusCode,  
      mensaje: mensaje,  
      stack: err.stack  
    })  
  } else {  
    res.status(statusCode).send({ mensaje: mensaje })  
  }  
}  
  
module.exports = errorHandler
```

7. En la carpeta `middlewares` agregue un archivo llamado `bitacora.middleware.js` con el siguiente código.

```
const requestIp = require('request-ip');
const ClaimTypes = require('../config/claimtypes')
const { bitacora } = require('../models')

const bitacoralogger = (req, res, next) => {
  // Obtiene la IP de la petición
  const ip = requestIp.getClientIp(req)
  let email = 'invitado'

  req.bitacora = async (accion, id) => {
    if( req.decodedToken ){
      // Obtiene el email de usuario
      email = req.decodedToken[ClaimTypes.Name];
    }

    // Guarda la operacion
    await bitacora.create({
      accion: accion, elementoid: id, ip: ip, usuario: email, fecha: new Date()
    })
  }
  next()
}

module.exports = bitacoralogger
```

8. En la carpeta `middlewares` agregue un nuevo archivo llamado `upload.middleware.js` con el siguiente código que contendrá esta funcionalidad.

```
const multer = require("multer");

// Filtro para recibir unicamente imagenes JPG
const imageFilter = (req, file, cb) => {
  if (file.mimetype.startsWith("image/jpeg") && file.originalname.endsWith(".jpg")) {
    cb(null, true);
  } else {
    cb("Solo se permiten imágenes con extensión JPG.", false);
  }
};

// se configura el almacenamiento para los archivos subidos
var storage = multer.diskStorage({
  destination: (req, file, cb) => {
    cb(null, "uploads/");
  },
  filename: (req, file, cb) => {
    cb(null, `${Date.now()}-${file.originalname}`);
  },
});

// Se crea la instancia de multer
var uploadFile = multer({ storage: storage, fileFilter: imageFilter });
module.exports = uploadFile;
```

9. Más adelante se agregarán más middlewares referentes a la autenticación y autorización de usuarios.
10. Enhorabuena, ha realizado correctamente el apartado.

Instrucciones. Agregar los Controladores.

Controlador: Categorías

1. Agregue un nuevo archivo llamado `categorias.controller.js` en la carpeta `controllers` con el siguiente código.

```
const { categoria } = require('../models')
const { body, validationResult } = require('express-validator');

let self = {}

self.categoriaValidator = [
  |   body('nombre', 'El campo {0} es obligatorio').not().isEmpty()
  | ]

// GET: api/categorias
> self.getAll = async function (req, res, next) { ...
  | }

// GET: api/categorias/5
> self.get = async function (req, res, next) { ...
  | }

// POST: api/categorias
> self.create = async function (req, res, next) { ...
  | }

// PUT: api/categorias/5
> self.update = async function (req, res, next) { ...
  | }

// DELETE: api/categorias/5
> self.delete = async function (req, res, next) { ...
  | }

module.exports = self
```

2. Cada línea marcada con azul, es un método que iremos creando paso a paso.
3. Agregue el código del método `getAll()`.

```
// GET: api/categorias
self.getAll = async function (req, res, next) {
  try {
    let data = await categoria.findAll({ attributes: [['id', 'categoriaId'], 'nombre', 'protegida'] })
    res.status(200).json(data)
  } catch (error) {
    next(error)
  }
}
```

4. Agregue el código del método `get()`.

```
// GET: api/categorias/5
self.get = async function (req, res, next) {
  try {
    let id = req.params.id
    let data = await categoria.findPk(id, { attributes: [['id', 'categoriaId'], 'nombre', 'protegida'] })
    if (data)
      res.status(200).json(data)
    else
      res.status(404).send()
  } catch (error) {
    next(error)
  }
}
```

5. Agregue el código del método `create()`.

```
// POST: api/categorias
self.create = async function (req, res, next) {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) throw new Error(JSON.stringify(errors));

    let data = await categoria.create({
      nombre: req.body.nombre
    })
    // Bitacora
    req.bitacora("categoria.crear", data.id)
    res.status(201).json(data)
  } catch (error) {
    next(error)
  }
}
```

6. Agregue el código del método `update()`.

```
// PUT: api/categorias/5
self.update = async function (req, res, next) {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) throw new Error(JSON.stringify(errors));

    let id = req.params.id
    let body = req.body
    let data = await categoria.update(body, { where: { id: id } })
    if (data[0] === 0)
      return res.status(404).send()

    // Bitacora
    req.bitacora("categoria.editar", id)
    res.status(204).send()
  } catch (error) {
    next(error)
  }
}
```

7. Agregue el código del método `delete()`.

```
// DELETE: api/categorias/5
self.delete = async function (req, res, next) {
  try {
    const id = req.params.id
    let data = await categoria.findByPk(id)
    if (!data)
      return res.status(404).send()
    // No se pueden eliminar categorias protegidas
    if (data.protegida)
      return res.status(400).send()

    data = await categoria.destroy({ where: { id: id } })
    if (data === 1) {
      // Bitacora
      req.bitacora("categoria.eliminar", id)
      return res.status(204).send()
    }
    res.status(404).send()
  } catch (error) {
    next(error)
  }
}
```

Controlador: Productos

1. Agregue un nuevo archivo llamado `productos.controller.js` en la carpeta `controllers` con el siguiente código.

```
const { producto, categoria, Sequelize } = require('../models')
const { body, validationResult } = require('express-validator');
const Op = Sequelize.Op

let self = {}

self.productoValidator = [
  body('titulo', 'El campo {0} es obligatorio').not().isEmpty(),
  body('descripcion', 'El campo {0} es obligatorio').not().isEmpty(),
  body('precio', 'El campo {0} es obligatorio').not().isEmpty().isDecimal({ force_decimal: false }),
]

// GET: api/productos
self.getAll = async function (req, res, next) { ...
}

// GET: api/productos/5
self.get = async function (req, res, next) { ...
}

// POST: api/productos
self.create = async function (req, res, next) { ...
}

// PUT: api/productos/5
self.update = async function (req, res, next) { ...
}

// DELETE: api/productos/5
self.delete = async function (req, res, next) { ...
}

// POST: api/productos/5/categoria
self.asignaCategoria = async function (req, res, next) { ...
}

// DELETE: api/productos/5/categoria/1
self.eliminaCategoria = async function (req, res, next) { ...
}

module.exports = self
```

-
2. Cada línea marcada con azul, es un método que iremos creando paso a paso.
3. Agregue el código del método `getAll()`.

```
// GET: api/productos
self.getAll = async function (req, res, next) {
  try {
    // Recibe el parametro de busqueda
    const { s } = req.query

    // Filtro para buscar
    const filters = {}
    if (s) {
      filters.titulo = {
        [Op.like]: `'%${s}%'`
      }
    }

    let data = await producto.findAll({
      where: filters,
      attributes: [['id', 'productoId'], 'titulo', 'descripcion', 'precio', 'archivoid'],
      include: {
        model: categoria,
        as: 'categorias',
        attributes: [['id', 'categoriaId'], 'nombre', 'protegida'],
        through: { attributes: [] }
      },
      subQuery: false
    })
    return res.status(200).json(data)
  } catch (error) {
    next(error)
  }
}
```

-
-
-
4. Agregue el código del método `get()`.

```
// GET: api/productos/5
self.get = async function (req, res, next) {
  try {
    let id = req.params.id
    let data = await producto.findByPk(id, {
      attributes: [['id', 'productoId'], 'titulo', 'descripcion', 'precio', 'archivoid'],
      include: {
        model: categoria,
        as: 'categorias',
        attributes: [['id', 'categoriaId'], 'nombre', 'protegida'],
        through: { attributes: [] }
      }
    })
    if (data)
      res.status(200).json(data)
    else
      res.status(404).send()
  } catch (error) {
    next(error)
  }
}
```


5. Agregue el código del método `create()`.

```
// POST: api/productos
self.create = async function (req, res, next) {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) throw new Error(JSON.stringify(errors));

    let data = await producto.create({
      titulo: req.body.titulo,
      descripcion: req.body.descripcion,
      precio: req.body.precio,
      archivoid: req.body.archivoid || null
    })
    // Bitacora
    req.bitacora("producto.crear", data.id)
    res.status(201).json(data)
  } catch (error) {
    next(error)
  }
}
```

6. Agregue el código del método `update()`.

```
// PUT: api/productos/5
self.update = async function (req, res, next) {
  try {
    const errors = validationResult(req)
    if (!errors.isEmpty()) throw new Error(JSON.stringify(errors));

    let id = req.params.id
    let body = req.body
    let data = await producto.update(body, { where: { id: id } })
    if (data[0] === 0)
      return res.status(404).send()
    // Bitacora
    req.bitacora("producto.editar", id)
    res.status(204).send()
  } catch (error) {
    next(error)
  }
}
```

7. Agregue el código del método `delete()`.

```
// DELETE: api/productos/5
self.delete = async function (req, res, next) {
  try {
    let id = req.params.id
    let data = await producto.findByPk(id)
    if (!data)
      return res.status(404).send()

    data = await producto.destroy({ where: { id: id } })
    if (data === 1) {
      // Bitacora
      req.bitacora("producto.eliminar", id)
      return res.status(204).send()
    }
    res.status(404).send()
  } catch (error) {
    next(error)
  }
}
```

8. Agregue el código del método `asignaCategoria()`.

```
// POST: api/productos/5/categoria
self.asignaCategoria = async function (req, res, next) {
  try {
    let itemToAssign = await categoria.findByPk(req.body.categoriaid);
    if (!itemToAssign) return res.status(404).send()

    let item = await producto.findByPk(req.params.id);
    if (!item) return res.status(404).send()

    await item.addCategoria(itemToAssign)
    // Bitacora
    req.bitacora("productocategoria.agregar", `${req.params.id}:${req.body.categoriaid}`)
    res.status(204).send()
  } catch (error) {
    next(error)
  }
}
```

9. Agregue el código del método `eliminaCategoria()`.

```
// DELETE: api/productos/5/categoria/1
self.eliminaCategoria = async function (req, res, next) {
  try {
    let itemToRemove = await categoria.findByPk(req.params.categoriaid);
    if (!itemToRemove) return res.status(404).send()

    let item = await producto.findByPk(req.params.id);
    if (!item) return res.status(404).send()

    await item.removeCategoria(itemToRemove)
    // Bitacora
    req.bitacora("productocategoria.remover", `${req.params.id}:${req.params.categoriaid}`)
    res.status(204).send()
  } catch (error) {
    next(error)
  }
}
```

Controlador: Archivos

1. Agregue un nuevo archivo llamado `archivos.controller.js` en la carpeta `controllers` con el siguiente código.

```
const { archivo } = require('../models')
const fs = require("fs")

let self = {}

// GET: api/archivos
self.getAll = async function (req, res, next) { ...
}

// GET: api/archivos/5/detalle
self.getDetalle = async function (req, res, next) { ...
}

// GET: api/archivos/5
self.get = async function (req, res, next) { ...
}

// POST: api/archivos
self.create = async function (req, res, next) { ...
}

// PUT: api/archivos/5
self.update = async function (req, res, next) { ...
}

// DELETE: api/archivos/5
self.delete = async function (req, res, next) { ...
}

module.exports = self
```

2. Cada línea marcada con azul, es un método que iremos creando paso a paso.
3. Agregue el código del método `getAll()`.

```
// GET: api/archivos
self.getAll = async function (req, res, next) {
  try {
    let data = await archivo.findAll({ attributes: [['id', 'archivoId'], 'mime', 'indb', 'nombre', 'size'] })
    res.status(200).json(data)
  } catch (error) {
    next(error)
  }
}
```

4. Agregue el código del método `getDetalle()`.

```
// GET: api/archivos/5/detalle
self.getDetalle = async function (req, res, next) {
  try {
    let id = req.params.id
    let data = await archivo.findByPk(id, { attributes: [['id', 'archivoId'], 'mime', 'indb', 'nombre', 'size'] })
    if (data)
      res.status(200).json(data)
    else
      res.status(404).send()
  } catch (error) {
    next(error)
  }
}
```

5. Agregue el código del método `get()`.

```
// GET: api/archivos/5
self.get = async function (req, res, next) {
  try {
    let id = req.params.id
    let data = await archivo.findByPk(id)
    if (!data)
      return res.status(404).send()

    let imagen = data.datos
    if (!data.indb)
      imagen = fs.readFileSync("uploads/" + data.nombre)

    res.status(200).contentType(data.mime).send(imagen)
  } catch (error) {
    next(error)
  }
}
```

6. Agregue el código del método `create()`.

```
// POST: api/archivos
self.create = async function (req, res, next) {
  try {
    console.log(req.file)
    if (req.file == undefined) return res.status(400).json('El archivo es obligatorio.');

    let binario = null;
    let indb = false;
    if (process.env.FILES_IN_BD == "true") {
      binario = fs.readFileSync("uploads/" + req.file.filename)
      fs.existsSync("uploads/" + req.file.filename) && fs.unlinkSync("uploads/" + req.file.filename)
      indb = true;
    }
    // Crea el registro
    let data = await archivo.create({
      mime: req.file.mimetype,
      indb: indb,
      nombre: req.file.filename,
      size: req.file.size,
      datos: binario
    })

    // Bitacora
    req.bitacora("archivos.crear", data.id)
    //Envia la respuesta
    res.status(201).json({
      id: data.id,
      mime: req.file.mimetype,
      indb: indb,
      nombre: req.file.filename
    })
  } catch (error) {
    next(error)
  }
}
```

7. Agregue el código del método `update()`.

```
// PUT: api/archivos/5
self.update = async function (req, res, next) {
  try {
    if (req.file == undefined) return res.status(400).json('El archivo es obligatorio.');

    let id = req.params.id
    let imagen = await archivo.findById(id)
    if (!imagen) {
      fs.existsSync("uploads/" + req.file.filename) && fs.unlinkSync("uploads/" + req.file.filename)
      return res.status(404).send()
    }

    let binario = null;
    let indb = false;
    if (process.env.FILES_IN_BD == "true") {
      binario = fs.readFileSync("uploads/" + req.file.filename)
      fs.existsSync("uploads/" + req.file.filename) && fs.unlinkSync("uploads/" + req.file.filename)
      indb = true;
    }
    // Modifica el registro
    let data = await archivo.update({
      mime: req.file.mimetype,
      indb: indb,
      nombre: req.file.filename,
      size: req.file.size,
      datos: binario
    }, { where: { id: id } })

    // Bitacora
    req.bitacora("archivos.editar", data.id)
    //Envia la respuesta
    if (data[0] === 0)
      return res.status(404).send()

    // Eliminamos el archivo anterior
    if (!imagen.indb)
      fs.existsSync("uploads/" + imagen.nombre) && fs.unlinkSync("uploads/" + imagen.nombre)

    res.status(204).send()
  } catch (error) {
    next(error)
  }
}
```

8. Agregue el código del método `delete()`.

```
// DELETE: api/archivos/5
self.delete = async function (req, res, next) {
  try {
    const id = req.params.id
    let imagen = await archivo.findByPk(id)
    if (!imagen)
      return res.status(404).send()

    let data = await archivo.destroy({ where: { id: id } })
    if (data === 1) {
      // Bitacora
      req.bitacora("archivos.eliminar", id)
      // Eliminamos el archivo anterior
      if (!imagen.indb)
        fs.existsSync("uploads/" + imagen.nombre) && fs.unlinkSync("uploads/" + imagen.nombre)
      return res.status(204).send()
    }
    res.status(404).send()
  } catch (error) {
    next(error)
  }
}
```

Controlador: Usuarios

1. Agregue un nuevo archivo llamado `usuarios.controller.js` en la carpeta `controllers` con el siguiente código.

```
const { usuario, rol, Sequelize } = require('../models')
const bcrypt = require('bcrypt')
const crypto = require('crypto')

let self = {}

// GET: api/usuarios
self.getAll = async function (req, res, next) { ...
}

// GET: api/usuarios/email
self.get = async function (req, res, next) { ...
}

// POST: api/usuarios
self.create = async function (req, res, next) { ...
}

// PUT: api/usuarios/email
self.update = async function (req, res, next) { ...
}

// DELETE: api/usuarios/email
self.delete = async function (req, res, next) { ...
}

module.exports = self
```

2. Cada línea marcada con azul, es un método que iremos creando paso a paso.
3. Agregue el código del método `getAll()`.

```
// GET: api/usuarios
self.getAll = async function (req, res, next) {
  try {
    const data = await usuario.findAll({
      raw: true,
      attributes: ['id', 'email', 'nombre', [Sequelize.col('rol.nombre'), 'rol']],
      include: { model: rol, attributes: [] }
    })
    res.status(200).json(data)
  } catch (error) {
    next(error)
  }
}
```

4. Agregue el código del método `get()`.

```
// GET: api/usuarios/email
self.get = async function (req, res, next) {
  try {
    const email = req.params.email
    const data = await usuario.findOne({
      where: { email: email },
      raw: true,
      attributes: ['id', 'email', 'nombre', [Sequelize.col('rol.nombre'), 'rol']],
      include: { model: rol, attributes: [] }
    })
    if (data)
      return res.status(200).json(data)
    res.status(404).send()
  } catch (error) {
    next(error)
  }
}
```

5. Agregue el código del método `create()`.


```
// POST: api/usuarios
self.create = async function (req, res, next) {
  try {
    const rolusuario = await rol.findOne({ where: { nombre: req.body.rol } })

    const data = await usuario.create({
      id: crypto.randomUUID(),
      email: req.body.email,
      passwordhash: await bcrypt.hash(req.body.password, 10),
      nombre: req.body.nombre,
      rolid: rolusuario.id
    })

    // Bitacora
    req.bitacora("usuarios.crear", data.email)
    res.status(201).json({
      id: data.id,
      email: data.email,
      nombre: data.nombre,
      rolid: rolusuario.nombre
    })
  } catch (error) {
    next(error)
  }
}
```

6. Agregue el código del método `update()`.

```
// PUT: api/usuarios/email
self.update = async function (req, res, next) {
  try {
    const email = req.params.email
    const rolusuario = await rol.findOne({ where: { nombre: req.body.rol } })
    req.body.rolid = rolusuario.id

    const data = await usuario.update(req.body, {
      where: { email: email },
    })

    if (data[0] === 0)
      return res.status(404).send()

    // Bitacora
    req.bitacora("usuarios.editar", email)
    res.status(204).send()
  } catch (error) {
    next(error)
  }
}
```

7. Agregue el código del método `delete()`.

```
// DELETE: api/usuarios/email
self.delete = async function (req, res, next) {
  try {
    const email = req.params.email
    let data = await usuario.findOne({ where: { email: email } })
    // No se pueden eliminar usuarios protegidos
    if (data.protegido) return res.status(403).send()

    data = await usuario.destroy({ where: { email: email } })
    if (data === 1) {
      // Bitacora
      req.bitacora("usuarios.eliminar", email)
      return res.status(204).send() // Elemento eliminado
    }

    res.status(403).send()
  } catch (error) {
    next(error)
  }
}
```

Controlador: Roles

1. Agregue un nuevo archivo llamado `roles.controller.js` en la carpeta `controllers` con el siguiente código.

```
const { rol } = require('../models')

let self = {}

// GET: api/roles
self.getAll = async function (req, res) {
  let data = await rol.findAll({ attributes: ['id', 'nombre'] })
  res.status(200).json(data)
}

module.exports = self
```

Controlador: Bitácora

1. Agregue un nuevo archivo llamado `bitacora.controller.js` en la carpeta `controllers` con el siguiente código.

```
const { bitacora } = require('../models')

let self = {}

// GET: api/bitacora
self.getAll = async function (req, res, next) {
  let data = await bitacora.findAll({
    attributes: [['id', 'bitacoraId'], 'accion', 'elementoid', 'ip', 'usuario', 'fecha'],
    order: [['id', 'DESC']],
  })
  res.status(200).json(data)
}

module.exports = self
```

2. Enhorabuena, ha creado sus controladores de manera correcta.

Instrucciones. Agregar autenticación a su API

JSON Web Token ayuda a proteger una ruta de un usuario no autenticado. El uso de JWT en la aplicación evitará que los usuarios no autenticados accedan a **endpoints** protegidos.

JWT crea un token, lo envía al cliente y, a continuación, el cliente utiliza el token para realizar solicitudes. También ayuda a verificar que sea un usuario válido que realiza esas solicitudes.

1. Se tiene que instalar el soporte para JWT antes de usarlo en la aplicación.

```
npm install jsonwebtoken --save
```

2. Ahora se tiene que generar una llave privada llamada **secret** para poder generar los JWT.
3. Desde la línea de comandos genere una cadena aleatoria de 35 bytes en hexadecimal. Entre a la línea de comandos de **node**.

```
node
Welcome to Node.js v20.11.0.
Type ".help" for more information.
>
```

4. Y luego escriba este comando para generar la cadena.

```
> require("crypto").randomBytes(35).toString("hex")
```

```
PS C:\codigo\tw\backendnodeauth> node
Welcome to Node.js v20.11.0.
Type ".help" for more information.
> require("crypto").randomBytes(35).toString("hex")
'3ca4407f27a00a1b628b9f1416356e6d799b6b6a93f820a6098b560df066f5537a5ccb'
> |
```

5. Para salir de la línea de comandos de **node** presione dos veces **Ctrl + C**.
6. Copie y pegue esa cadena en su archivo **.env** en la variable **JWT_SECRET**.

```
DB_HOST=localhost
DB_DATABASE=mercadolibrenode
DB_USER=mercadolibre_user
DB_PASSWORD=M3rc@d0
DB_PORT=3306
SERVER_PORT=3000
NODE_ENV=development
JWT_SECRET=865ad24cf96fa78ab5c3cdb48f5fb114923d95c8a4e4f1bc9c33f6eeda32ed9d4b8c57
FILES_IN_BD=false
```

7. No comparta esta cadena secreta. Si se filtra esta cadena secreta, los usuarios no autenticados pueden crear tokens falsos para acceder a las rutas.

Instrucciones. Agregar servicios.

1. Sus clientes van a requerir tokens de acceso, así que tenemos que crear un servicio que pueda generarlos.
2. Agregue un nuevo archivo llamado `jwttoken.service.js` en la carpeta `services` con el siguiente código.

```
const jwt = require('jsonwebtoken')
const jwtSecret = process.env.JWT_SECRET
const ClaimTypes = require('../config/claimtypes')

const GeneraToken = (email, nombre, rol) => {
  // Utilizamos los nombres de Claims estandar
  const token = jwt.sign({
    [ClaimTypes.Name]: email,
    [ClaimTypes.GivenName]: nombre,
    [ClaimTypes.Role]: rol,
    "iss": "ServidorFeiJWT",
    "aud": "ClientesFeiJWT"
  },
    jwtSecret, {
    expiresIn: '20m', // 20 minutos
  })
  return token;
}

const TiempoRestanteToken = (req) => {
  try {
    const authHeader = req.header('Authorization')
    // Obtiene el token de la solicitud
    const token = authHeader.split(' ')[1]
    // Verifica el token, si no es válido envía error y salta al catch
    const decodedToken = jwt.verify(token, jwtSecret)

    // Regresa el tiempo restante en minutos
    const time = (decodedToken.exp - (new Date().getTime() / 1000))
    const minutos = Math.floor(time / 60)
    const segundos = Math.floor(time - minutos * 60)
    return "00:" + minutos.toString().padStart(2, "0") + ':' + segundos.toString().padStart(2, "0")
  } catch (error) {
    return null
  }
}

module.exports = { GeneraToken, TiempoRestanteToken }
```

3. Es obligatorio colocar el `ClaimTypes.Name` y `ClaimTypes.Role` para que plataformas como .NET, sepan en cada petición, el nombre del usuario y su rol asignado. Cada uno esa basado en un esquema estándar.

ClaimTypes.Name	http://schemas.xmlsoap.org/ws/2005/05/identity/claims/name
ClaimTypes.GivenName	http://schemas.xmlsoap.org/ws/2005/05/identity/claims/givenname
ClaimTypes.Role	http://schemas.microsoft.com/ws/2008/06/identity/claims/role

- [illegible]

- The image shows a web application for decoding JWT tokens. The interface is divided into two main panels: 'Encoded' and 'Decoded'. The 'Encoded' panel on the left contains a long string of Base64-encoded characters. The 'Decoded' panel on the right shows the decoded structure of the token, including the header, payload, and signature. The header identifies the token as a JWT using the HS256 algorithm. The payload contains claims for 'alg', 'typ', 'iss', 'exp', 'iat', and 'sub'. The signature is a long Base64 string. The interface also includes a top navigation bar with links like 'Debugger', 'Libraries', 'Introduction', and 'Ask', and a bottom status bar indicating the token is 'PASTE A TOKEN HERE'.

Encoded PASTE A TOKEN HERE

```
eyJhbGciOiJIodHRwOi8vd3d3LnczLm9yZy8yMDA  
xLzA0L3htbGRzaWctbW9yZSNOBWFjLXNOYTI1Ni  
IsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1h  
cy54bWxzZ2FwLm9yZy93cy8yMDA1LzA1L2lkZW5  
0aXR5L2NsYWltcy9uYW11Ijoiz3ZlcmFAdXYubX  
giLClodHRwOi8vc2NoZW1hcy54bWxzZ2FwLm9yZ  
y93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9n  
aXZlbm5hbWUiOiJHdWlsbGVyYbW8gSHVtYmVydG8  
gVmVyYSBBbWFybyIsImh0dHA6Ly9zY2h1bWZzLm  
1pY3Jvc29mdC5jb20vd3MvMjAwOC8wNi9pZGVud  
GI0eS9jbGFpbXMvc9sZSI6IkFkbWluaXN0cmFk  
b3IiLClJeHAiOiJ0eS9jbGFpbXMvc9sZSI6IkFkbW  
lcnZpZG9yRmVpSldUIiwiYXVkiIjoizxpZW50ZX  
NGZWlKV1Q1fQ.z0AFQg93P2APPsgC5F0HZA89mf  
Xb9UEaKvssild0tnI
```

Decoded EDIT THE PAYLOAD AND SECRET

HEADER: ALGORITHM & TOKEN TYPE

```
{  
  "alg": "http://www.w3.org/2001/04/xmldsig-more#hmac-  
sha256",  
  "typ": "JWT"  
}
```

PAYLOAD: DATA

```
{  
  
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/n  
ame": "gvera@uv.mx",  
  
  "http://schemas.xmlsoap.org/ws/2005/05/identity/claims/g  
ivenname": "Guillermo Humberto Vera Amaro",  
  
  "http://schemas.microsoft.com/ws/2008/06/identity/claims  
/role": "Administrador",  
  "exp": 1713590036,  
  "iss": "ServidorFeiJWT",  
  "aud": "ClientesFeiJWT"  
}
```

- El token es válido y observe que puede ver el usuario que tiene el acceso. Solo con esa cadena token, usted puede tener acceso al sistema por 20 minutos. Es por ello que **JWT** no se considera un método de acceso tan estricto, pues si este token es robado, cualquiera podría acceder a los recursos.
- Observe que por último, se agrega la función para el cálculo del tiempo restante del token generado.
- Enhorabuena, ha realizado correctamente este apartado Sus clientes deberán entrar a la ruta `/api/auth/` para iniciar sesión en la API y obtener un token válido.

Instrucciones. Agregar el inicio de sesión: Autenticación.

1. En la carpeta `controllers`, cree un nuevo archivo llamado `auth.controller.js` con el código

```
const bcrypt = require('bcrypt')
const { usuario, rol, Sequelize } = require('../models')
const { GeneraToken, TiempoRestanteToken } = require('../services/jwttoken.service')

let self = {}

// POST: api/auth
self.login = async function (req, res, next) {
  const { email, password } = req.body

  try {
    let data = await usuario.findOne({
      where: { email: email },
      raw: true,
      attributes: ['id', 'email', 'nombre', 'passwordhash', [Sequelize.col('rol.nombre'), 'rol']],
      include: { model: rol, attributes: [] }
    })

    if (data === null)
      return res.status(401).json({ mensaje: 'Usuario o contraseña incorrectos.' })

    // Se compara la contraseña vs el hash almacenado
    const passwordMatch = await bcrypt.compare(password, data.passwordhash)
    if (!passwordMatch)
      return res.status(401).json({ mensaje: 'Usuario o contraseña incorrectos.' })

    // Utilizamos los nombres de Claims estandar
    token = GeneraToken(data.email, data.nombre, data.rol)

    // Bitacora
    req.bitacora("usuario.login", data.email)

    res.status(200).json({
      email: data.email,
      nombre: data.nombre,
      rol: data.rol,
      jwt: token
    })
  } catch (error) {
    next(error)
  }
}

// GET: api/auth/tiempo
self.tiempo = async function (req, res) {
  const tiempo = TiempoRestanteToken(req)
  if (tiempo == null)
    res.status(404).send()

  res.status(200).send(tiempo)
}

module.exports = self
```

2. Lo que se busca es que sus clientes deban entrar a la ruta `/api/auth/` para iniciar sesión en la API y obtener un token válido.

Instrucciones. Agregar middleware para la autorización

Sus clientes van a enviarle sus tokens de acceso en cada petición. Hay que validarlos. Además van requerir tokens de acceso no expiren mientras los están utilizando. Así que agregue la funcionalidad para refrescar la validez del token si el usuario haciendo peticiones al sistema.

1. Para esto se implementará un middleware que verifique la validez del token enviado por el cliente y también se verificará si el rol enviado está autorizado para realizar la acción solicitada. Por último si el token está cerca de expirar, se le enviará uno nuevo en un encabezado llamado **Set-Authorization**.
2. En la carpeta **middlewares** agregue un nuevo archivo llamado **auth.middleware.js** con el siguiente código que contendrá esta funcionalidad.

```
const jwt = require('jsonwebtoken')
const jwtSecret = process.env.JWT_SECRET
const ClaimTypes = require('../config/claimtypes')
const { GeneraToken } = require('../services/jwttoken.service')

const Authorize = (rol) => {
  return async (req, res, next) => {
    try {
      const authHeader = req.header('Authorization')
      const error = new Error('Acceso denegado');
      error.statusCode = 401;
      if (!authHeader.startsWith('Bearer '))
        return next(error);

      // Obtiene el token de la solicitud
      const token = authHeader.split(' ')[1]
      // Verifica el token, si no es válido envía error y salta al catch
      const decodedToken = jwt.verify(token, jwtSecret)

      // Verifica si el rol está autorizado
      if (rol.split(',').indexOf(decodedToken[ClaimTypes.Role]) == -1)
        return next(error);

      // Si tiene acceso, se permite continuar con el método y se obtienen los datos del usuario
      req.decodedToken = decodedToken

      // Código para enviar un nuevo token
      var minutosRestantes = (decodedToken.exp - (new Date().getTime() / 1000)) / 60;
      //Si quedan 5 minutos, le mandamos un nuevo token
      if (minutosRestantes < 5) {
        var nuevoToken = GeneraToken(decodedToken[ClaimTypes.Name], decodedToken[ClaimTypes.GivenName], decodedToken[ClaimTypes.Role])
        res.header("Set-Authorization", nuevoToken)
      }

      // Continúa con el método
      next()
    } catch (error) {
      error.statusCode = 401
      next(error)
    }
  }
}

module.exports = Authorize
```

3. Enhorabuena, ha realizado correctamente el apartado.

Instrucciones. Agregar las rutas.

Cuando un cliente realice una petición por medio de un **endpoint** de HTTP (GET, POST, PUT, DELETE), necesitamos determinar cómo responderá el servidor estableciendo las rutas.

1. Dentro de la carpeta **routes** agrega un nuevo archivo llamado **categorias.routes.js** con código.

```
const router = require('express').Router()
const categorias = require('../controllers/categorias.controller')
const Authorize = require('../middlewares/auth.middleware')

// GET: api/categorias
router.get('/', Authorize('Usuario,Administrador'), categorias.getAll)

// GET: api/categorias/5
router.get('/:id', Authorize('Usuario,Administrador'), categorias.get)

// POST: api/categorias
router.post('/', Authorize('Administrador'), categorias.categoriaValidator, categorias.create)

// PUT: api/categorias/5
router.put('/:id', Authorize('Administrador'), categorias.categoriaValidator, categorias.update)

// DELETE: api/categorias/5
router.delete('/:id', Authorize('Administrador'), categorias.delete)

module.exports = router
```

2. En este archivo establecemos que la ruta **/api/categorias** será manejada por el controlador **/controllers/categorias.controller.js**.
3. Dentro de la carpeta **routes** agrega un nuevo archivo llamado **productos.routes.js** con el código.

```
const router = require('express').Router()
const productos = require('../controllers/productos.controller')
const Authorize = require('../middlewares/auth.middleware')

// GET: api/productos
router.get('/', Authorize('Usuario,Administrador'), productos.getAll)

// GET: api/productos/5
router.get('/:id', Authorize('Usuario,Administrador'), productos.get)

// POST: api/productos
router.post('/', Authorize('Administrador'), productos.productoValidator, productos.create)

// PUT: api/productos/5
router.put('/:id', Authorize('Administrador'), productos.productoValidator, productos.update)

// DELETE: api/productos/5
router.delete('/:id', Authorize('Administrador'), productos.delete)

// POST: api/productos/5/categoria
router.post('/:id/categoria', Authorize('Administrador'), productos.asignaCategoria);

// DELETE: api/productos/5/categoria
router.delete('/:id/categoria/:categoriaid', Authorize('Administrador'), productos.eliminaCategoria);

module.exports = router
```

4. En este archivo establecemos que la ruta `/api/productos` será manejada por el controlador `/controllers/productos.controller.js`.
5. Dentro de la carpeta `routes` agrega un nuevo archivo llamado `usuarios.routes.js` con código.

```
const router = require('express').Router()
const usuarios = require('../controllers/usuarios.controller')
const Authorize = require('../middlewares/auth.middleware')

// GET: api/usuarios
router.get('/', Authorize('Administrador'), usuarios.getAll)

// GET: api/usuarios/email
router.get('/:email', Authorize('Administrador'), usuarios.get)

// POST: api/usuarios
router.post('/', Authorize('Administrador'), usuarios.create)

// PUT: api/usuarios/email
router.put('/:email', Authorize('Administrador'), usuarios.update)

// DELETE: api/usuarios/email
router.delete('/:email', Authorize('Administrador'), usuarios.delete)

module.exports = router
```

6. En este archivo establecemos que la ruta `/api/usuarios` será manejada por el controlador `/controllers/usuarios.controller.js`.
7. Dentro de la carpeta `routes` agrega un nuevo archivo llamado `roles.routes.js` con código.

```
const router = require('express').Router()
const roles = require('../controllers/roles.controller')
const Authorize = require('../middlewares/auth.middleware')

// GET: api/roles
router.get('/', Authorize('Administrador'), roles.getAll)

module.exports = router
```

8. En este archivo establecemos que la ruta `/api/roles` será manejada por el controlador `/controllers/roles.controller.js`.

9. Observe como se están protegiendo todos los métodos de los controladores que van a requerir autenticación para ser accedidos. Esto se realiza agregando el middleware de autorización para cada ruta o controlador que la aplicación requiera proteger. Por ejemplo, observe que en su archivo de `/routes/categorias.routes.js` en cada `endpoint` se está colocando la invocación del middleware `Authorize()` antes de la llamada a método del controlador. Si el middleware no envía un error de no autorizado, la función se ejecutará por la función `next()` dentro del middleware. Si desea que dos o más roles pudieran acceder a ese `endpoint`, solo agregue el nombre del rol separado por comas en el método que implementa el `endpoint`.
10. Dentro de la carpeta `routes` agrega un nuevo archivo llamado `archivos.routes.js` con código.

```
const router = require('express').Router()
const archivos = require('../controllers/archivos.controller')
const Authorize = require('../middlewares/auth.middleware')
const upload = require("../middlewares/upload.middleware")

// GET: api/archivos
router.get('/', Authorize('Administrador'), archivos.getAll)

// GET: api/archivos/5
router.get('/:id', archivos.get)

// GET: api/archivos/5/detalle
router.get('/:id/detalle', Authorize('Administrador'), archivos.getDetalle)

// POST: api/archivos
router.post('/', upload.single("file"), Authorize('Administrador'), archivos.create)

// PUT: api/archivos/5
router.put('/:id', upload.single("file"), Authorize('Administrador'), archivos.update)

// DELETE: api/archivos/5
router.delete('/:id', Authorize('Administrador'), archivos.delete)

module.exports = router
```

11. Dentro de la carpeta `routes` agrega un nuevo archivo llamado `auth.routes.js` con código.

```
const router = require('express').Router()
const auth = require('../controllers/auth.controller')
const Authorize = require('../middlewares/auth.middleware')

// POST: api/auth
router.post('/', auth.login)

// GET: api/auth/tiempo
router.get('/tiempo', Authorize('Usuario,Administrador'), auth.tiempo)

module.exports = router
```

12. Dentro de la carpeta `routes` agrega un nuevo archivo llamado `bitacora.routes.js` con código.

```
const router = require('express').Router()
const bitacora = require('../controllers/bitacora.controller')
const Authorize = require('../middlewares/auth.middleware')

// GET: api/bitacora
router.get('/', Authorize('Administrador'), bitacora.getAll)

module.exports = router
```

13. Enhorabuena, ha creado las rutas de su aplicación.

Instrucciones. Configurar su programa principal

1. Abra su archivo `index.js` y elimine todo el código en él. Agregue el siguiente código.

```
const express = require('express')
const cors = require("cors")
const dotenv = require('dotenv')
const app = express()

// Primero carga la configuración del archivo .env para que este disponible en las demás llamadas
dotenv.config()

// Se requiere para entender los datos recibidos en JSON
app.use(express.json())
app.use(express.urlencoded({ extended: false }))

// Se requiere si se accede desde un navegador web
var corsOptions = {
  origin: ["http://localhost:8080", "http://localhost:8080"],
  methods: "GET,PUT,POST,DELETE",
}
app.use(cors(corsOptions))

// Swagger
const swaggerUi = require('swagger-ui-express')
const swaggerFile = require('./swagger-output.json')
app.use('/swagger', swaggerUi.serve, swaggerUi.setup(swaggerFile))

// Bitacora
app.use(require('./middlewares/bitacora.middleware'))

// Rutas
app.use("/api/categorias", require('./routes/categorias.routes'))
app.use("/api/productos", require('./routes/productos.routes'))
app.use("/api/usuarios", require('./routes/usuarios.routes'))
app.use("/api/roles", require('./routes/roles.routes'))
app.use("/api/auth", require('./routes/auth.routes'))
app.use("/api/archivos", require('./routes/archivos.routes'))
app.use("/api/bitacora", require('./routes/bitacora.routes'))
app.get('*', (req, res) => { res.status(404).send("Recurso no encontrado") })

// Middleware para el manejo de errores (Debe ser el último middleware a utilizar)
const errorHandler = require('./middlewares/errorhandler.middleware')
app.use(errorHandler)

// Inicia el servidor web en el puerto SERVER_PORT
app.listen(process.env.SERVER_PORT, () => {
  console.log(`Aplicación de ejemplo escuchando en el puerto ${process.env.SERVER_PORT}`)
})
```

2. Lea los comentarios que se proveen, le explican el funcionamiento de cada instrucción.
3. Enhorabuena, su aplicación esta lista para trabajar.

Instrucciones. Agregar documentación a su API

1. Hay varias maneras de generar una documentación automática para su API. Generalmente esto se hace para apoyar a los desarrolladores de frontend y sepan cómo se pueden comunicar con sus *endpoints*.
2. Node.js genera una API compatible con la especificación **OpenAPI** por lo que pueden utilizarse generadores de documentación de API como **Swagger**.
3. Para agregar **Swagger** a su proyecto, instale el siguiente paquete desde la terminal de Visual Studio Code.

```
npm install swagger-autogen --save-dev
npm install swagger-ui-express --save
```

4. Ahora cree un nuevo archivo en la raíz de su proyecto llamado `swagger.js` con el siguiente contenido.

```
const swaggerAutogen = require('swagger-autogen')()

const doc = {
  info: {
    // Nombre del API
    title: 'Backend Node.js API',
    description: 'Esta es una API en node.js'
  },
  host: 'localhost:3000'
}

// Se generará un nuevo archivo con la documentación
const outputFile = './swagger-output.json'
const routes = ['./index.js']

// Se genera la documentación
swaggerAutogen(outputFile, routes, doc)
```

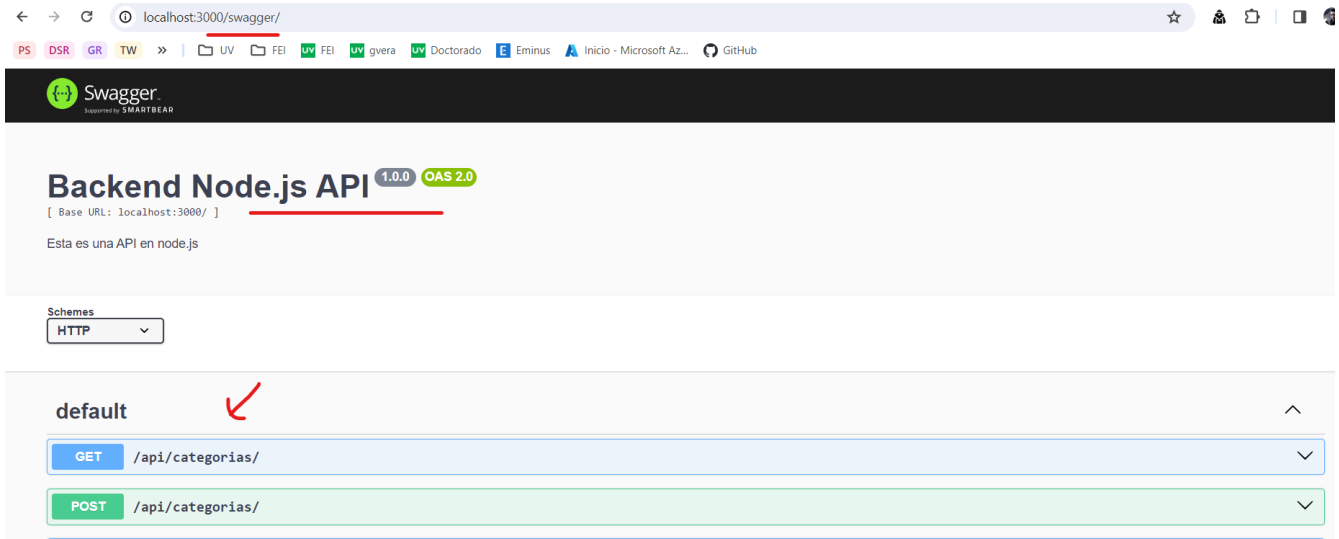
5. Ahora ejecute el siguiente comando para generar el archivo `swagger-output.json` en raíz.

```
node swagger.js
```

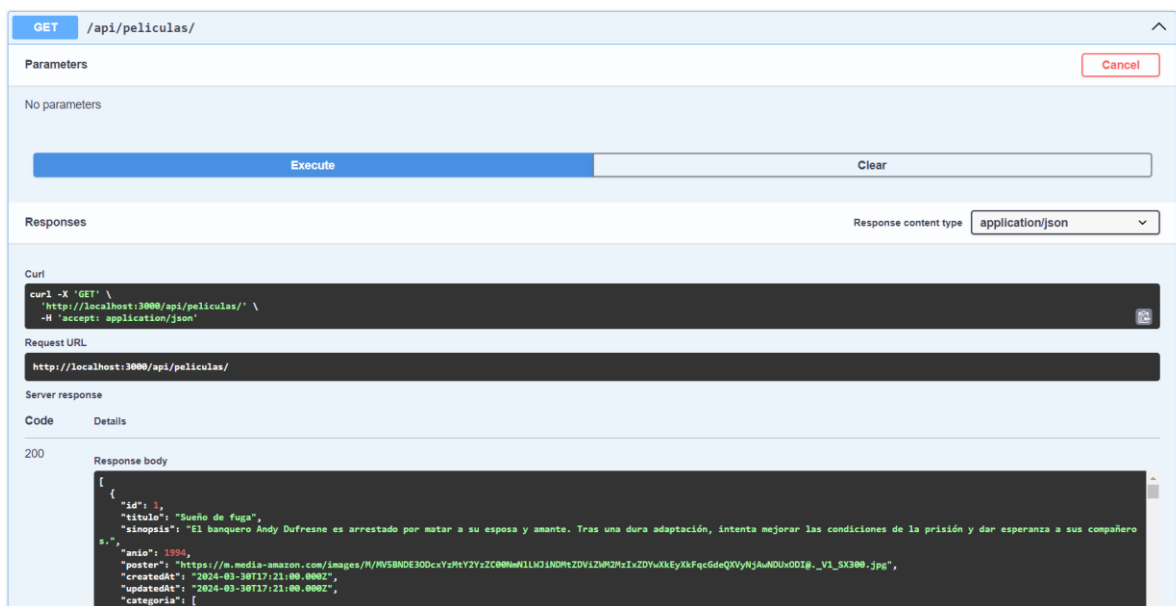
6. Ahora abra su archivo principal de su proyecto `index.js` y debajo de la funcionalidad de controladores, asegúrese de incluir el servicio de generación de documentación de **Swagger**. Por último, antes de las rutas agregue la interfaz de **Swagger** y que se pueda acceder desde la ruta `/swagger` de su aplicación web.

```
// Swagger
const swaggerUi = require('swagger-ui-express')
const swaggerFile = require('./swagger-output.json')
app.use('/swagger', swaggerUi.serve, swaggerUi.setup(swaggerFile))
```

7. Vuelva a ejecutar su aplicación y ahora cuando entre a la ruta `/swagger`, verá la interfaz de Swagger donde se documentan los *endpoints* expuestos por su API.



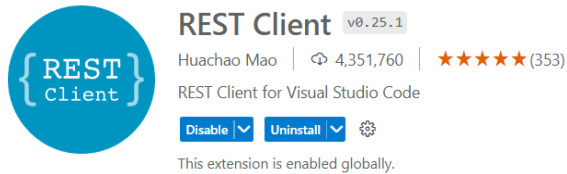
8. Puede utilizar esta interfaz también para probar el funcionamiento de su API. Navegue en ella y verifique que todo funciona correctamente.



9. Felicidades, ha creado su aplicación de manera correcta.

Instrucciones. Probar su API

1. Utilizando su navegador web no es la forma más práctica de probar una API. Generalmente se utiliza otro tipo de clientes web para hacer las pruebas de manera más eficiente.
2. Asegúrese de tener instalada la extensión **REST Client** para Visual Studio Code.



3. Agregue una nueva carpeta llamada `restclient` y dentro un nuevo archivo llamado `auth.http` en la raíz de su proyecto con el siguiente código.

```
@token = aquivaeltoken

GET http://localhost:3000/api/categorias
Authorization: Bearer {{ token }}

###

POST http://localhost:3000/api/auth
Content-Type: application/json

{
  "email": "gvera@uv.mx",
  "password": "patito"
}

###

POST http://localhost:3000/api/auth
Content-Type: application/json

{
  "email": "patito@uv.mx",
  "password": "patito"
}

###

GET http://localhost:3000/api/auth/tiempo
Authorization: Bearer {{ token }}

###

GET http://localhost:3000/api/roles
Content-Type: application/json
Authorization: Bearer {{ token }}

###
```


- ```
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 1359
ETag: W/"54f-NH+ce8nODVenihAz34G5BU0w9GY"
Date: Tue, 13 Aug 2024 02:34:52 GMT
Connection: close
```

- ```
POST http://localhost:3000/api/auth
Content-Type: application/json

{
  "email": "gvera@uv.mx",
  "password": "patito"
}
```

- ```
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 585
ETag: W/"249-VwmYvD8YSVIBlcPkfauPFdqEK/U"
Date: Tue, 13 Aug 2024 02:38:11 GMT
Connection: close
```

```
{
 "email": "gvera@uv.mx",
 "nombre": "Guillermo Vera",
 "rol": "Administrador",
 "jwt": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzbnFwLm9yZy93cy8yMDAxLzA1L2lkZW50aXR5L2NsYWltcy9uYVllIjoiaz3ZlcmFadXYubXgiLCJodHRwOi8vc2NoZW1hcy54bWxzbnFwLm9yZy93cy8yMDAxLzA1L2lkZW50aXR5L2NsYWltcy9naXZlbm5hbWUiOiJHdWlsbGVybnw8GmVYYSIsImh0dHA6L29yZ2hlbnFzLm1pY3Jvc29mdC5jb20vdDZMvMjAwOC8wNi9pZGVudGl0eS9jbG9pbXVmc9sZSI6IkdFkbWluaXN0cmFkb3IiLCJpc3MiOiJ1JTZJ2aWRvckZlaUpXVCIsImF1ZCI6IkdksaWVudGVzRmVpSlduIiwiaWF0IjoxNzIzNTE2NjksLCJleHAiOjE3MjM1MTC4OTF9.V17Sv17P0tVlRCrTyZ8QuCX05r0ttB1Cb5bV3s6M7E5"
}
```

- Si envía sus credenciales correctas, obtendrá un token de acceso con una validez de 20 minutos que puede adjuntar a sus peticiones como ticket de autenticación y autorización.
- Este token contiene su `email`, su `nombre` y su `rol`.

10. Ahora agregue esta petición a su archivo `auth.http`. Copie y pegue el token generado en el inicio de sesión después de la palabra `Bearer`. Respete el espacio colocado después de `Bearer`.

```
@token = eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzbn2FwLm9yZy93cy8

GET http://localhost:3000/api/categorias
Authorization: Bearer {{ token }}
```

11. Verifique ya puede obtener la lista de categorías.

```
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 924
ETag: W/"39c-NeyK7kYZzw2f/20UfmI+e70wVAs"
Date: Tue, 13 Aug 2024 02:40:21 GMT
Connection: close
```

```
✓ [
✓ {
 "categoriaId": 1,
 "nombre": "Televisores",
 "protegida": false
},
✓ {
 "categoriaId": 2,
 "nombre": "Celulares y tecnología",
 "protegida": false
},
]
```

12. Haga lo mismo con los otros `endpoints` como por ejemplo, obtener el tiempo restante.

|                                                                                                          |                                                                                                                                                                    |                                                                                                                                                                                                                                                    |
|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>Send Request GET http://localhost:3000/api/auth/tiempo Authorization: Bearer {{ token }}  ###</pre> | <pre>http://localhost:3000/api/auth/tiempo http://localhost:3000/api/auth/tiempo http://localhost:3000/api/auth/tiempo http://localhost:3000/api/auth/tiempo</pre> | <pre>1 HTTP/1.1 200 OK 2 X-Powered-By: Express 3 Vary: Origin 4 Content-Type: text/html; charset=utf-8 5 Content-Length: 8 6 ETag: W/"8-a6uMl/SOGiJYG6BC0tT8WHjRjdg" 7 Date: Tue, 13 Aug 2024 02:41:03 GMT 8 Connection: close 9 10 00:17:07</pre> |
|----------------------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|

13. Pruebe con un usuario que tenga el perfil de `Usuario` y verifique que no pueda acceder a los recursos del rol `Administrador`.

```
HTTP/1.1 401 Unauthorized
X-Powered-By: Express
Vary: Origin
Content-Type: application/json; charset=utf-8
Content-Length: 1338
ETag: W/"53a-ctRrmD7n5V6bw0IVeB3tdOmZx3M"
Date: Tue, 13 Aug 2024 02:45:51 GMT
Connection: close
```

```
{
 "status": 401,
 "mensaje": "Acceso denegado",
}
```

14. Al faltar 5 minutos para que su token expire, verifique sus peticiones devuelvan un nuevo token en el encabezado **Set-Authorization** a sus clientes para refrescar la autenticación.

```
HTTP/1.1 200 OK
X-Powered-By: Express
Vary: Origin
Set-Authorization: eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJodHRwOi8vc2NoZW1hcy54bWxzbnZlLnM9ZyZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW11IjoicGF0aXRvQHV2Lm14IiwiaHR0cDovL3NjaC54bWxzbnZlLnM9ZyZy93cy8yMDA1LzA1L2lkZW50aXR5L2NsYWltcy9uYW11IjoivXN1YXJpbyBwIj0iLCJodHRwOi8vc2NoZW1hcy5taWY2Ij0iLCJhdWQiOiJDbGllbnRlc0ZlaUpXVCIsImhhdCJ6MTcyMzUxNzYwMSwiZXhwIjoxNzIzNTE4ODAxZjQ.wwVZU76wRYWZOAOY6xsZcqtW5gaVVqAbofWt5b-iHo
Content-Type: text/html; charset=utf-8
Content-Length: 8
ETag: W/"8-8J78h88VKCTR56QXUnYw2/hS6qU"
Date: Tue, 13 Aug 2024 02:53:21 GMT
Connection: close
```

15. Puede agregar un nuevo archivo de peticiones de prueba para cada controlador, por ejemplo puede crear el archivo **categorias.http** en la carpeta **restclient** con el siguiente código.

```
@token = aquivaeltoken

GET http://localhost:3000/api/categorias
Authorization: Bearer {{ token }}

###

GET http://localhost:3000/api/categorias/2

###

POST http://localhost:3000/api/categorias
Authorization: Bearer {{ token }}
Content-Type: application/json

{
 "nombre": "Categoria de prueba"
}

###

PUT http://localhost:3000/api/categorias/16
Content-Type: application/json

{
 "categoriaid": 17,
 "nombre": "Categoria de prueba editada"
}

###

DELETE http://localhost:3000/api/categorias/20
```



```
POST http://localhost:3000/api/archivos
Content-Type: multipart/form-data; boundary=XXX
Authorization: Bearer {{ token }}

--XXX
Content-Disposition: form-data; name="file"; filename="producto1.jpg"
Content-Type: image/jpeg

< ./assets/producto1.jpg
--XXX--

###

PUT http://localhost:3000/api/archivos/3
Content-Type: multipart/form-data; boundary=XXX
Authorization: Bearer {{ token }}

--XXX
Content-Disposition: form-data; name="file"; filename="producto2.jpg"
Content-Type: image/jpeg

< ./assets/producto2.jpg
--XXX--

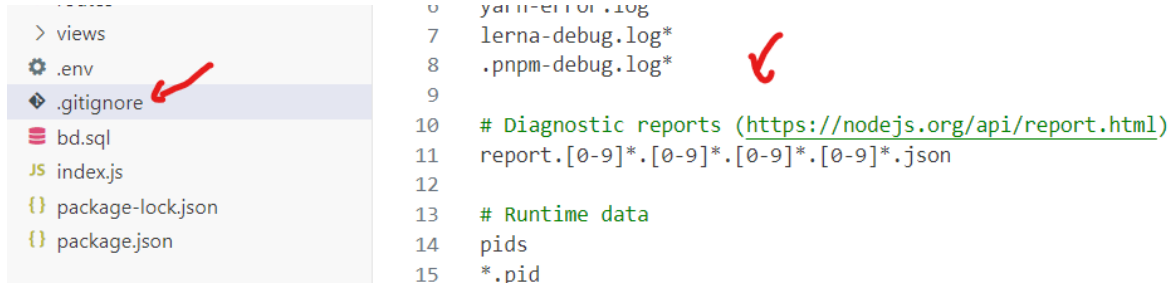
###

DELETE http://localhost:3000/api/archivos/1
Authorization: Bearer {{ token }}
```

25. Por último, pruebe su controlador de bitácora y que pueda obtener los registros almacenados.
26. Revise y reflexione sobre la estructura y funcionamiento de este Framework de desarrollo. La mayoría de frameworks para desarrollo web funcionan de una manera similar
27. Enhorabuena, su aplicación funciona y está protegida con autenticación y autorización.

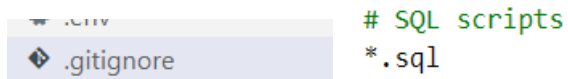
### Instrucciones. Colocar su aplicación a un repositorio de código fuente.

1. Por último, antes de subir su aplicación a GitHub, agregue en la raíz un nuevo archivo llamado `.gitignore` que previene el subir información innecesaria al repositorio público.
2. Copie el contenido de la plantilla que GitHub ofrece desde [Plantilla GitHub](#).
3. Este nuevo archivo aparece en la raíz de su proyecto.



4. Abra su archivo desde la raíz de su proyecto y agregue la carpeta `restclient` y la extensión `*.sql` para no subir estos elementos a GitHub pues pueden contener información sensible y privada.

```
REST Client
restclient
```



5. Observe como su archivo `.env` no se publicará en GitHub por lo que debe de proporcionarlo a sus clientes por algún otro medio que sea seguro.
6. Ahora ya puede publicar su archivo en GitHub para terminar la práctica.
7. Felicidades, ha creado su aplicación de manera correcta.